

《云计算技术》课程作业

云原生技术综合实验

从虚拟化到容器编排的消融对比

实 验 报 告

实验题目：云原生留言板的多层部署与消融对比

学 号：

姓 名：

实验平台：VirtualBox 7.1.4 / Docker 28.1.1 /
kind · K8s v1.31

代码仓库：github.com/LlsUlmus/hw4

完成日期：2026 年 10 月

目 录

一、实验概述	1
二、环境与工具	1
三、应用设计与实现	3
3.1 架构设计	3
3.2 核心接口	4
3.3 关键实现说明	5
3.4 容器化思路	5
四、部署过程记录	6
4.1 容器化	6
4.2 Kubernetes 部署与基础资源	10
4.3 自愈验证	13
4.4 弹性伸缩	15
4.5 进阶资源与高可用验证	19
4.6 Docker 与 K8s 的差异化比较	23
五、消融对比实验	26
5.1 对比路径与测试方法	26
5.2 启动时间	27
5.3 资源占用	29
5.4 性能	30
5.5 部署复杂度	34
5.6 隔离性	35
5.7 嵌套虚拟化	36
5.8 结论：何时选虚拟机，何时选容器	36
六、问题与分析	36
问题 1：虚拟机内无法拉取 Docker Hub 镜像	37
问题 2：虚拟机磁盘空间不足	37
问题 3：单容器测试中的两个细节	37
问题 4：就绪探针依赖 Redis，导致级联摘流	38
问题 5：Redis 已就绪，业务仍长时间连不上	38

问题 6: 缩容时偶发 502.....	40
问题 7: PDB 与 HPA 冲突, 节点永远无法排空	41
问题 8: CPU limit 让吞吐减半.....	41
问题 9: 节点宕机后约 40 秒的请求超时	41
问题 10: 嵌套虚拟机中 minikube 启动失败.....	42
问题 11: 其他值得记录的现象	42
七、心得体会与技能收获	42
7.1 我是什么时候真正分清容器和虚拟机的	42
7.2 K8s 带来的麻烦和便利	43
7.3 对“云原生”的新理解.....	43
7.4 如果重做一次	43
7.5 技能收获与仍存的疑问	44
八、参考文献与资料	44

一、实验概述

本次实验选择的场景是“微服务留言板”：用户在网页上发表留言、查看最新留言和访问统计。选择它，是因为它足够小、能在两周内做完，同时又天然包含无状态服务与有状态存储两类组件——前者适合拿来做多副本、扩缩容和自愈，后者能检验数据持久化与有状态服务的编排方式。应用被拆成三个运行单元：gateway（页面与 API 转发）、backend（留言、计数与实例统计业务）以及 Redis。每个接口都会在响应里带上处理请求的 gateway 和 backend 实例名，这样负载是否被分发、某个实例是否被替换，都可以直接从输出里看出来，而不必依赖推测。

技术栈方面，应用使用 Python 3.11 + Flask + gunicorn，存储使用 Redis 7；容器层使用 Docker 与 Docker Compose；编排层使用 kind 搭建的 1 控制面 + 2 工作节点 Kubernetes v1.31 集群，并补充了 metrics-server、ingress-nginx。宿主机是 Windows 11 家庭版笔记本，在其上用 VirtualBox 运行 Ubuntu 20.04 虚拟机，所有容器与集群都位于该虚拟机内；为了验证嵌套虚拟化，又在虚拟机内用 KVM 启动了第二层虚拟机来运行 minikube。

实验目标有三点：第一，完整打通“虚拟化 → 容器 → 编排 → 云原生应用”的链路，完成容器化、K8s 部署、自愈与弹性伸缩等验收项；第二，按“每次只增加一层”的思路，在物理机、虚拟机、Docker、K8s、嵌套虚拟机五种形态下部署同一个应用，用实测数据比较启动时间、资源占用、性能与隔离性；第三，记录实验中真实遇到的问题——本次共定位并修正了就绪探针引起的级联摘流、DNS 缓存导致的存储恢复延迟、缩容 502、PDB 与 HPA 冲突导致节点无法排空等问题，这些问题比“把环境跑起来”本身更能说明每一层技术的边界。

二、环境与工具

实验在一台笔记本上完成，整体分层如图 2-1 所示。宿主机未启用 WSL2 与 Hyper-V，且已有 VirtualBox 虚拟机可用，因此选择了“路径 A（Windows + VirtualBox 虚拟机内部署 Docker 与 K8s）”作为主路径，并在虚拟机内继续完成“路径 D（嵌套虚拟化部署 K8s）”；另以宿主机上直接运行进程作为最底层的基线。

同一应用逐层叠加部署：每次只增加一个技术层，观察启动时间、资源、性能与运维能力的变化

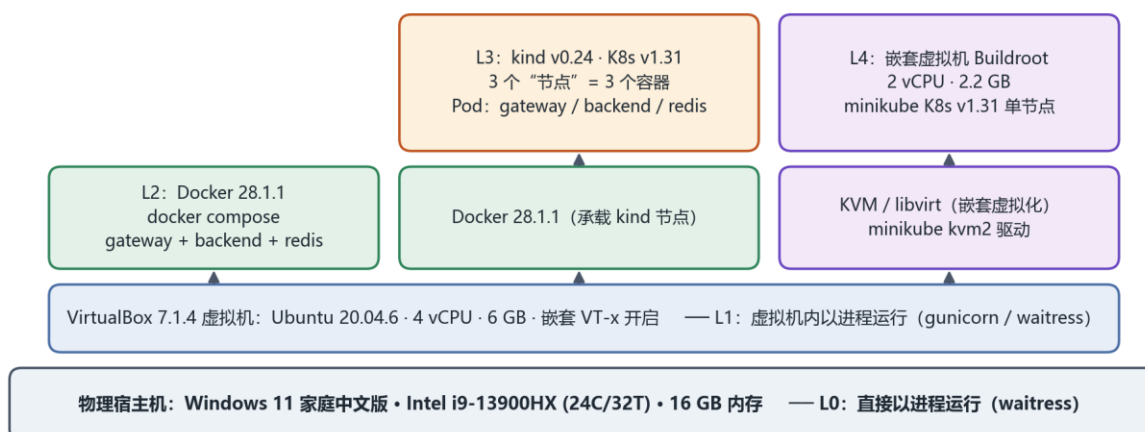


图 2-1 实验环境分层与消融路径

说明：L0~L4 为本实验逐层叠加的五种部署形态。Docker 与 kind 节点数据均放在挂载于 E 盘的独立虚拟磁盘（/data）上。

表 2-1 软硬件环境

层次	组件	版本 / 配置
宿主机	操作系统	Windows 11 家庭中文版 10.0.26200
	CPU / 内存	Intel Core i9-13900HX（24 核 32 线程）/ 16 GB
虚拟化	VirtualBox	7.1.4, 嵌套 VT-x 开启, NAT 端口转发
虚拟机	操作系统 / 内核	Ubuntu 20.04.6 LTS / 5.15.0-139-generic
	资源	4 vCPU, 6 GB 内存, 25 GB 系统盘 + 40 GB 数据盘
容器	Docker / Compose	Docker 28.1.1, Compose v2.35.1, cgroup v1, overlay2
编排	kubectl / kind	kubectl v1.31.14, kind v0.24.0（节点镜像 kindest/node:v1.31.0）
	集群插件	metrics-server v0.7.2, ingress-nginx v1.11.2
嵌套虚拟化	KVM / minikube	qemu-kvm + libvirt, minikube v1.34.0（kvm2 驱动, 2 vCPU / 2.2 GB）
应用	语言与框架	Python 3.11-slim, Flask 3.0.3, gunicorn 22.0, requests 2.32
	存储	Redis 7-alpine（AOF 持久化）
工具	压测与观测	自编 bench.py（标准库多线程）、probe.sh、docker stats、kubectl top、virsh domstats

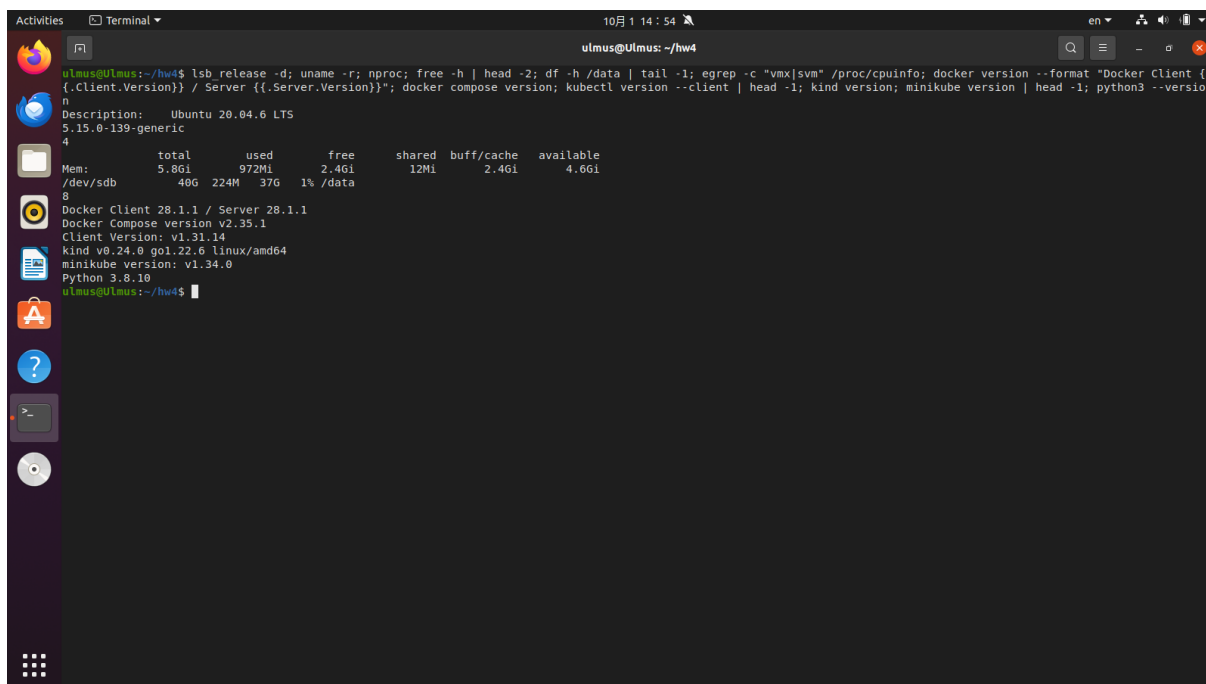


图 2-2 虚拟机内的系统与工具版本

说明：依次为系统版本、内核、vCPU 数、内存、数据盘、CPU 虚拟化标志数（8，说明嵌套 VT-x 已透传）以及 Docker、Compose、kubectl、kind、minikube、Python 版本。

三、应用设计与实现

3.1 架构设计

应用由三个运行单元组成，各自职责单一：gateway 负责提供前端页面并把 /api/* 请求转发到 backend，相当于一个极简的 API 网关；backend 承载全部业务逻辑；Redis 负责持久化。gateway 与 backend 都是无状态的，可以任意增减副本；Redis 是有状态的，在 K8s 中用 StatefulSet + PVC 部署。图 3-1 是最终版本在 K8s 中的资源关系。

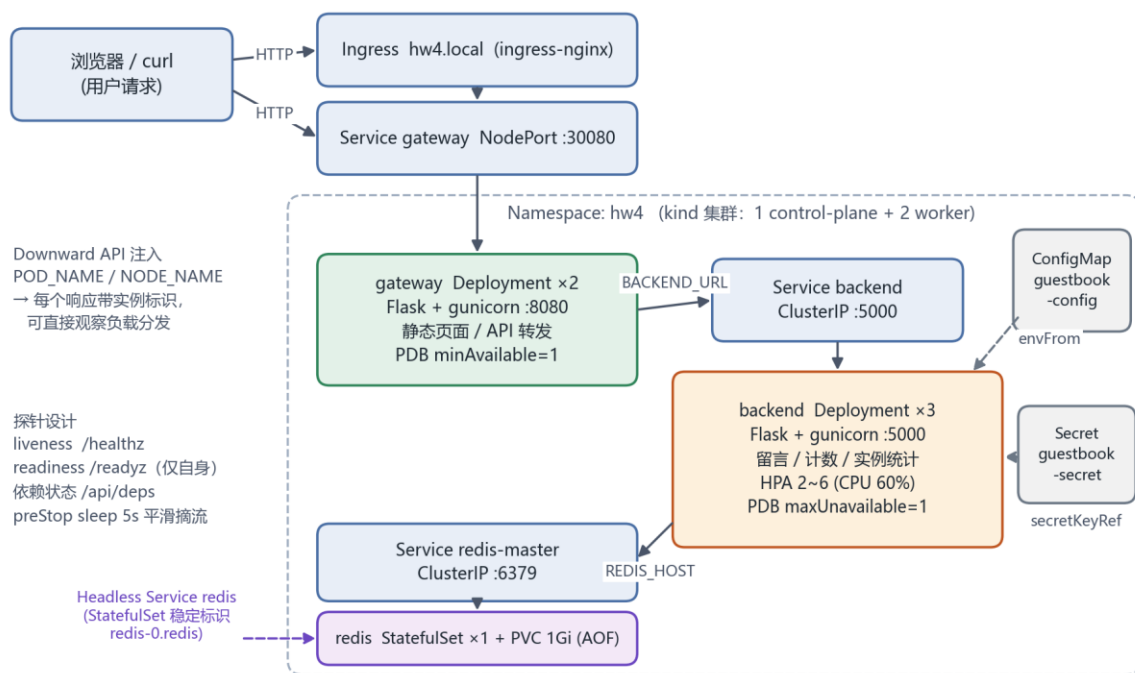


图 3-1 应用架构与 Kubernetes 资源关系

说明：外部流量经 Ingress 或 NodePort 进入 gateway，再经 ClusterIP Service 到达 backend；backend 通过 redis-master Service 访问 Redis。所有配置通过 ConfigMap / Secret 以环境变量注入，Pod 名与节点名通过 Downward API 注入。

3.2 核心接口

表 3-1 应用接口一览

方法	路径	所在服务	功能
GET	/	gateway	留言板页面（静态 HTML + 原生 JS）
GET	/api/whoami	gateway→backend	返回本次请求经过的 gateway、backend 实例及节点，用于观察负载分发
GET / POST	/api/messages	backend	查询最新留言 / 发表留言（长度校验、记录写入实例）
POST	/api/visit	backend	访问计数 +1，并记录由哪个实例处理
GET	/api/stats	backend	总访问量、留言总数、各实例命中次数
GET	/healthz	两者	存活探针：进程可响应即可
GET	/readyz	backend	就绪探针：只反映本实例能否处理请求（v1.1 修改）
GET	/api/deps	backend	依赖状态：Redis 是否可用

3.3 关键实现说明

(1) 实例标识。backend 优先读取环境变量 `POD_NAME` 作为实例名，读不到时退回容器 `hostname`。K8s 中通过 Downward API 注入 Pod 名和节点名，所以同一份代码在 Docker 中显示容器 ID，在 K8s 中显示 Pod 名，不需要任何分支逻辑：

```
INSTANCE = os.getenv("POD_NAME") or socket.gethostname()
NODE = os.getenv("NODE_NAME", "-")
```

(2) 存储抽象。backend 定义了接口一致的 `RedisStore` 与 `MemoryStore`，由环境变量 `STORAGE_MODE` 选择。内存模式只用于“物理机直接运行”这一消融基线，使基线测试不依赖 Redis，其余路径全部使用 Redis。

(3) 探针与依赖分离 (v1.1)。最初的 `/readyz` 会 ping Redis，实验中发现这会造成级联故障（详见第六章），因此改为只反映自身状态，Redis 不可用时只让用到存储的接口快速返回 503：

```
@app.get("/readyz")
def readyz():
    return jsonify({"status": "ready", "instance": INSTANCE})

@app.errorhandler(_redis.exceptions.RedisError)
def storage_unavailable(e):
    return jsonify({"error": "storage unavailable", "served_by": INSTANCE}), 503
```

3.4 容器化思路

两个服务各写一个 Dockerfile，构建上下文统一为仓库根目录。主要考虑如下：

- **基础镜像**：python:3.11-slim，镜像约 133 MB，比完整版小一个数量级，同时保留 glibc，避免 alpine 下编译依赖的问题。
- **分层缓存**：先 COPY requirements.txt 并安装依赖，再复制代码。改代码时依赖层命中缓存，实测重建只需数秒。
- **非 root 运行**：创建 uid 10001 的 appuser 并以其身份运行（图 4-3 中 id 命令可验证）。
- **健康检查**：Dockerfile 内置 HEALTHCHECK，compose 的 depends_on 与 docker ps 状态都能利用它。
- **配置外置**：端口、Redis 地址与口令、页面标题、版本号全部来自环境变量，镜像本身不含任何环境相关配置；PyPI 镜像源通过 ARG 传入，CI 中可覆盖。
- **进程模型**：使用 gunicorn 2 个 worker，取代 Flask 开发服务器。

```
FROM python:3.11-slim
```

```

ARG PIP_INDEX_URL=https://mirrors.aliyun.com/pypi/simple/
WORKDIR /app
COPY src/backend/requirements.txt .
RUN pip install -r requirements.txt          # 依赖层，可缓存
COPY src/backend/app.py .
RUN useradd --uid 10001 --no-create-home appuser
USER 10001
HEALTHCHECK --interval=10s CMD python -c "...urlopen('http://127.0.0.1:5000/healthz')"
CMD ["gunicorn", "-w", "2", "-b", "0.0.0.0:5000", "app:app"]

```

四、部署过程记录

本章按“容器化 → K8s 部署 → 自愈 → 弹性伸缩 → 进阶资源 → Docker 与 K8s 对比”的顺序记录。所有截图均取自虚拟机桌面终端，故障类实验采用三窗格布局：左上为 Pod 实时状态，右上为每 0.2 秒一次的持续访问探测（scripts/probe.sh），下方为执行的命令，从而在同一张图中同时呈现“操作—集群变化—服务可用性”。

4.1 容器化

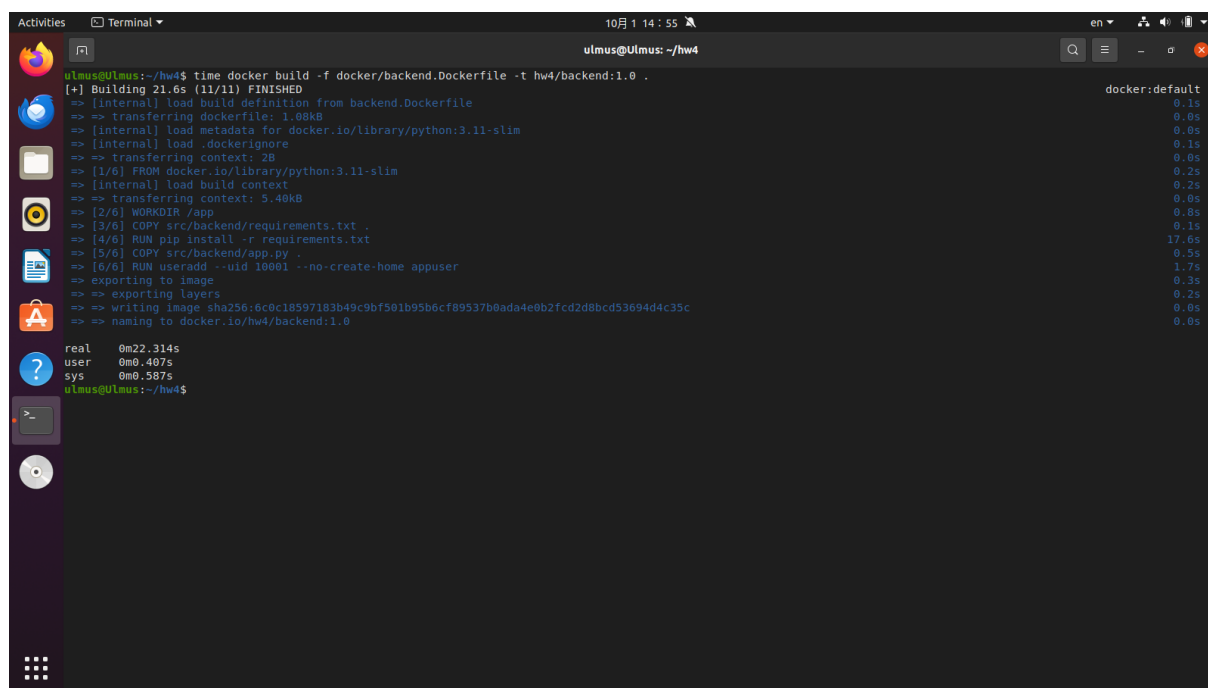


图 4-1 构建 backend 镜像（验证项：Dockerfile 可独立构建）

说明：首次构建耗时 22.3 s，其中 pip 安装依赖 17.6 s，占总时间的八成，这也是把依赖层单独缓存的原因。

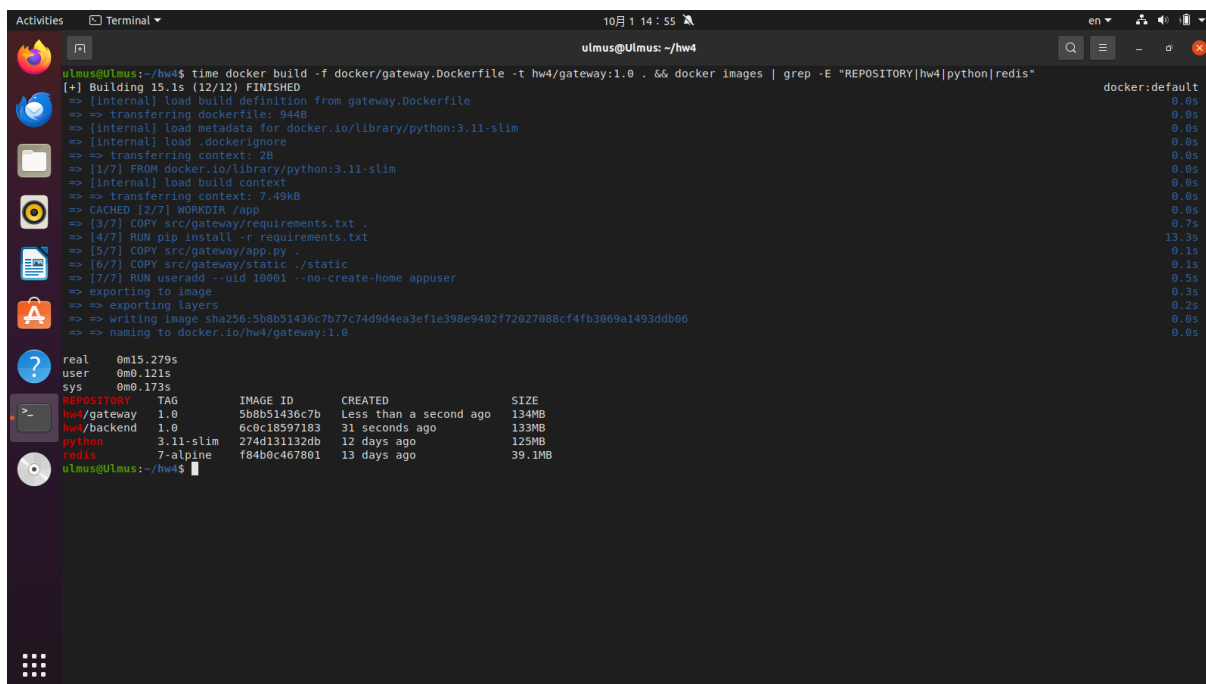


图 4-2 构建 gateway 镜像并查看本地镜像

说明：WORKDIR 层显示 CACHED，与 backend 共享基础层。两个应用镜像各约 134 MB，Redis 镜像 39 MB。

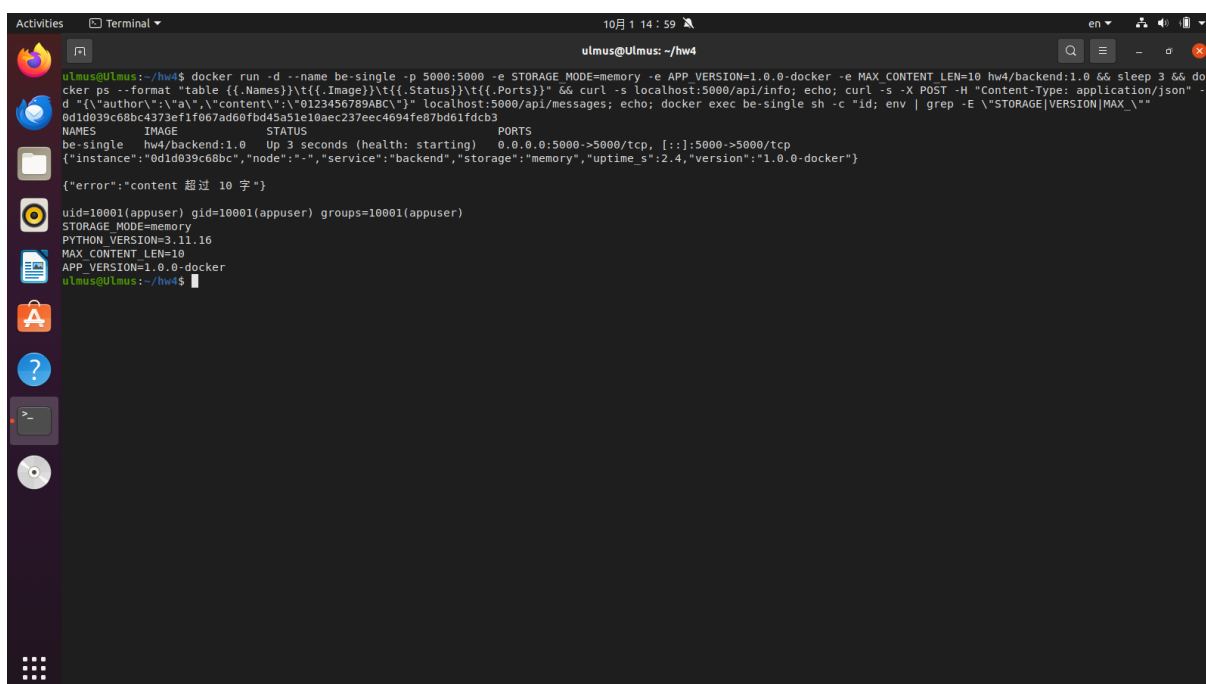


图 4-3 单容器运行与配置生效验证（验证项：端口、环境变量生效）

说明：以 -p 5000:5000 映射端口、以 -e 设置版本号与 MAX_CONTENT_LEN=10。/api/info 返回的 version 为 1.0.0-docker；提交 13 个字符的留言被拒绝并提示“content 超过 10 字”，说明环境变量确实改变了程序行为；id 命令显示进程以 uid 10001 运行。

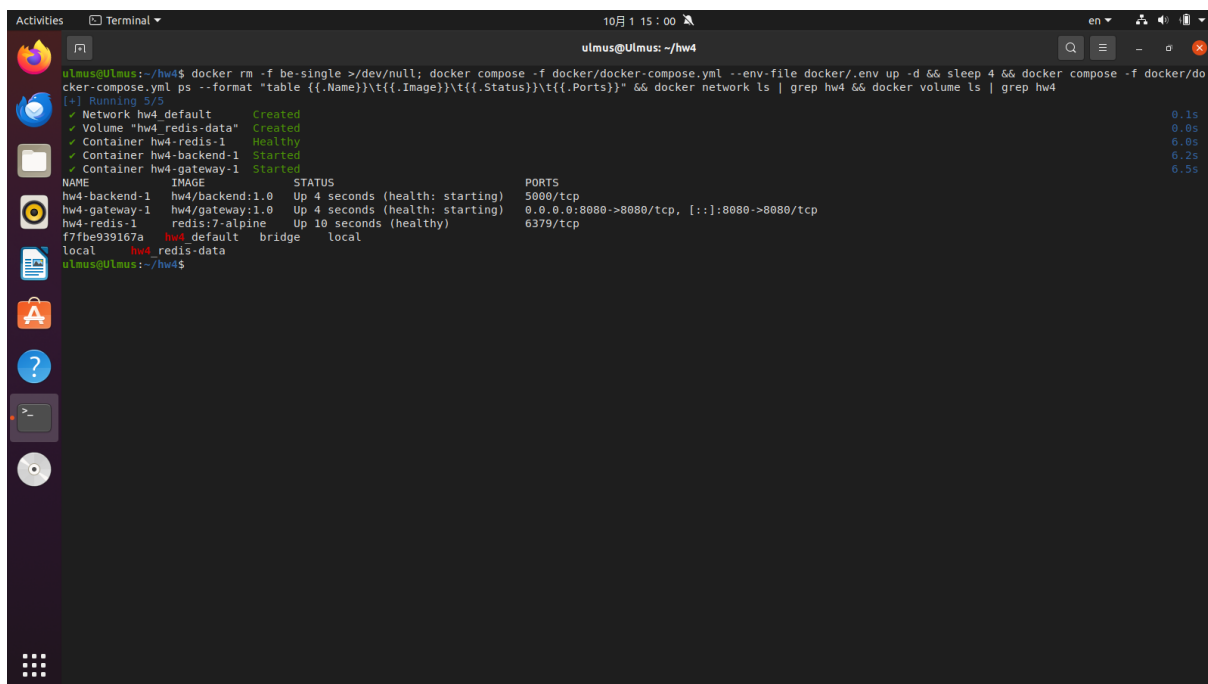


图 4-4 docker compose 启动三组件

说明：compose 自动创建 hw4_default 网络与 hw4_redis-data 数据卷；Redis 健康后才启动 backend（depends_on: service_healthy）。

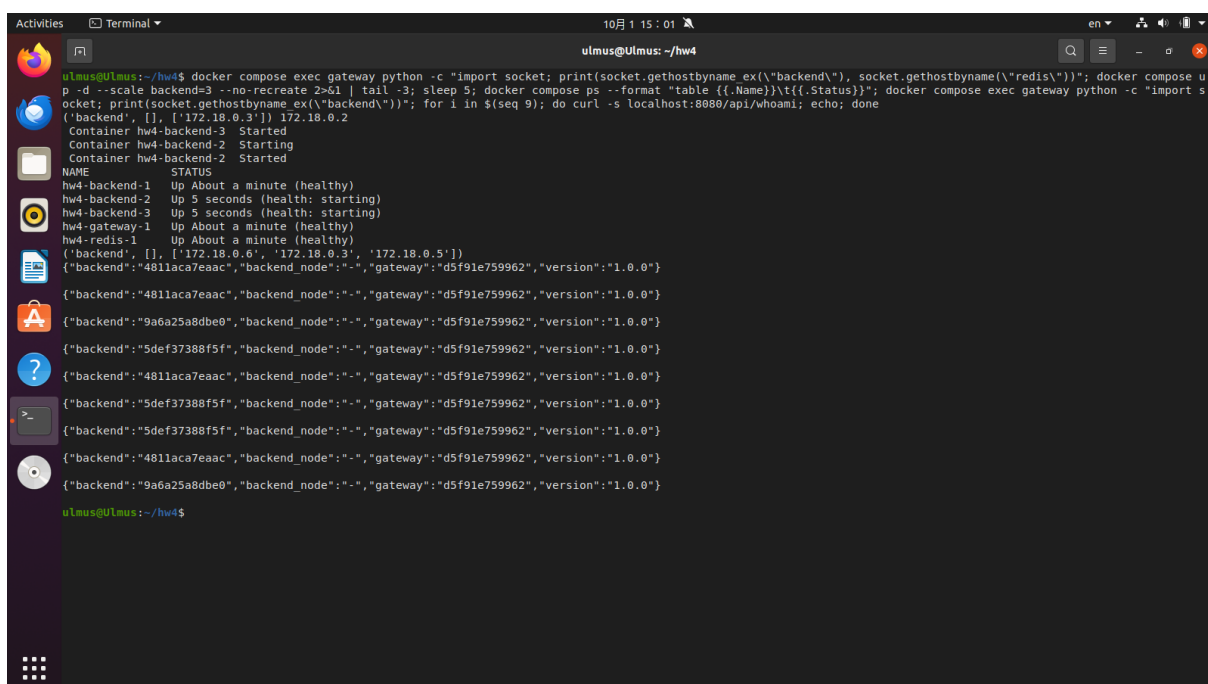


图 4-5 容器间通信与 backend 扩展为 3 副本（验证项：容器间通信）

说明：在 gateway 容器内解析服务名 backend 与 redis 均成功；扩展到 3 副本后，backend 解析出 3 个 IP，连续 9 次请求被分发到 3 个不同的 backend 容器（Docker 内置 DNS 轮询）。

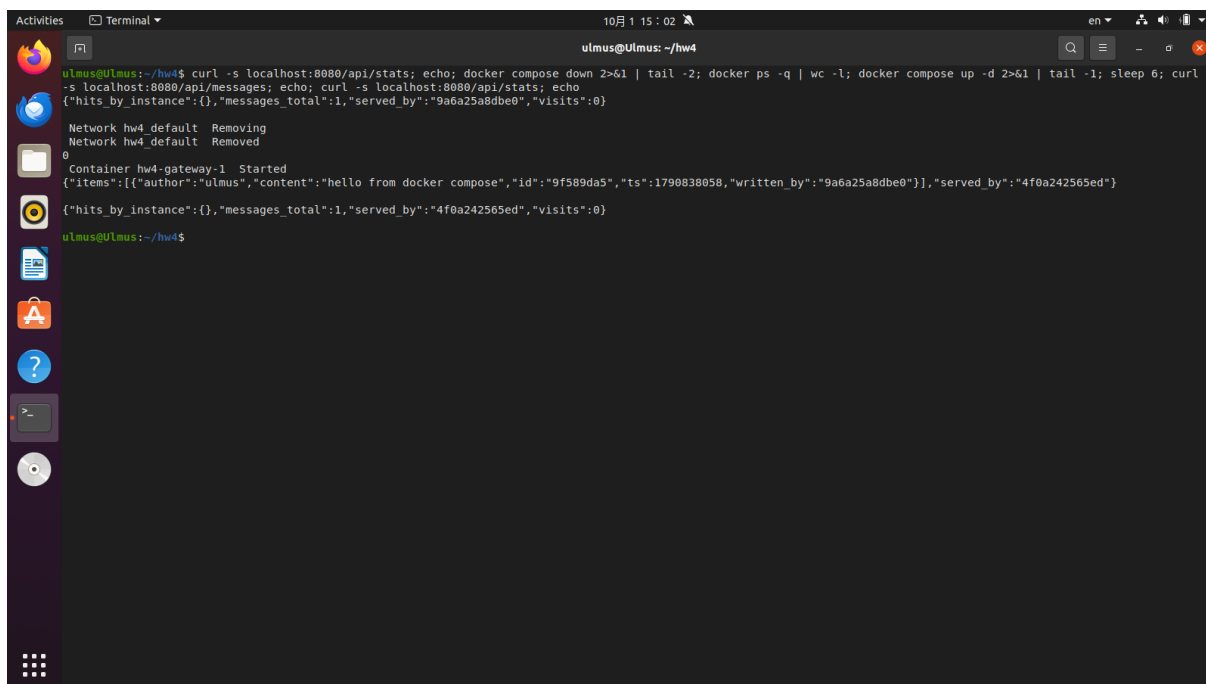


图 4-6 数据卷持久化验证

说明：docker compose down 后容器数为 0，再次 up 后此前写入的留言与计数仍在，数据保存在命名卷中而非容器内。同时可以注意到，up 之后 backend 回到了 1 个副本——compose 不会记住此前的 --scale。

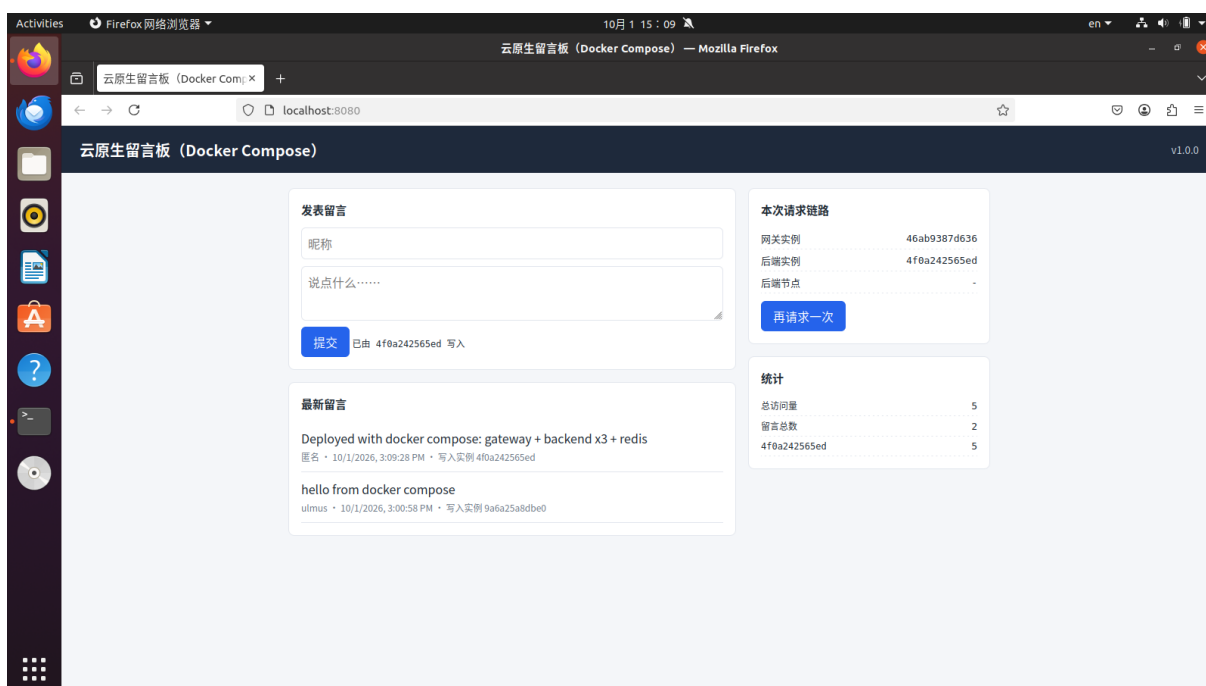


图 4-7 浏览器访问 compose 部署的留言板

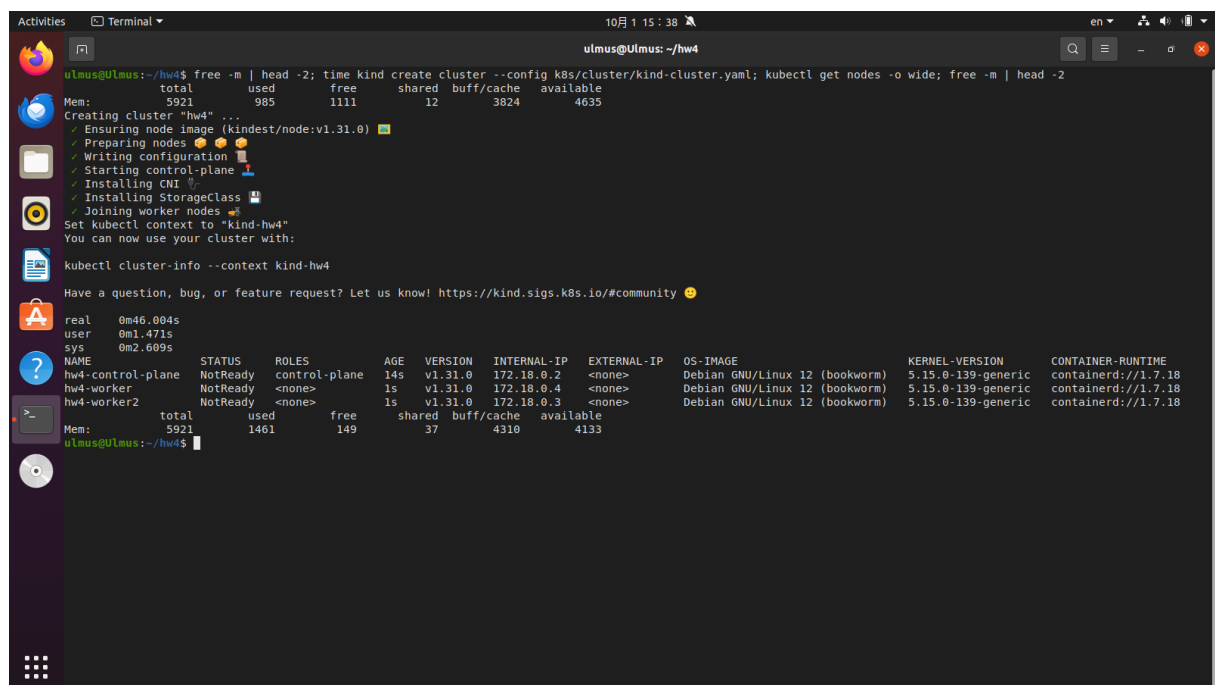
说明：页面右侧“本次请求链路”显示网关与后端容器 ID；通过页面提交的留言显示“已由 4f0a242565ed 写入”。

4.2 Kubernetes 部署与基础资源

集群使用 kind 创建，配置文件 k8s/cluster/kind-cluster.yaml 声明了 1 个控制面与 2 个工作节点，把 NodePort 30080 与 Ingress 的 80 端口映射到虚拟机，并为节点内的 containerd 配置了国内镜像源。资源清单按序号组织：

表 4-1 K8s 资源清单

文件	资源	要点
00-namespace.yaml	Namespace hw4	隔离实验资源
01-configmap.yaml	ConfigMap	标题、版本、BACKEND_URL、REDIS_HOST 等非敏感配置
02-secret.yaml	Secret	Redis 口令，以 secretKeyRef 注入
10-redis.yaml	Service ×2 + StatefulSet	Headless Service 提供稳定标识；redis-master 供业务访问；volumeClaimTemplates 申请 1Gi PVC
20-backend.yaml	Deployment + Service	3 副本、拓扑打散、就绪/存活探针、资源请求与限制、preStop、Downward API
30-gateway.yaml	Deployment + Service	2 副本、NodePort 30080
40-ingress.yaml	Ingress	hw4.local → gateway
50-pdb.yaml	PodDisruptionBudget ×2	backend maxUnavailable=1，gateway minAvailable=1
60-hpa.yaml	HorizontalPodAutoscaler	backend 2~6 副本，CPU 目标 60%



```
ulmus@ulmus:~/hw4$ free -m | head -2; time kind create cluster --config k8s/cluster/kind-cluster.yaml; kubectl get nodes -o wide; free -m | head -2
Mem:           5921      985      1111      12      3824      4635
Creating cluster "hw4" ...
  ✓ Ensuring node image (kindest/node:v1.31.0)
  ✓ Preparing nodes
  ✓ Writing configuration
  ✓ Starting control-plane
  ✓ Installing CNI
  ✓ Installing StorageClass
  ✓ Joining worker nodes
Set kubectl context to "kind-hw4"
You can now use your cluster with:

kubectl cluster-info --context kind-hw4

Have a question, bug, or feature request? Let us know! https://kind.sigs.k8s.io/#community
real    0m46.084s
user    0m1.471s
sys     0m2.609s
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
hw4-control-plane   NotReady  control-plane   14s   v1.31.0   172.18.0.2    <none>         Debian GNU/Linux 12 (bookworm) 5.15.0-139-generic containerd://1.7.18
hw4-worker          NotReady  <none>         1s    v1.31.0   172.18.0.4    <none>         Debian GNU/Linux 12 (bookworm) 5.15.0-139-generic containerd://1.7.18
hw4-worker2         NotReady  <none>         1s    v1.31.0   172.18.0.3    <none>         Debian GNU/Linux 12 (bookworm) 5.15.0-139-generic containerd://1.7.18
Mem:           5921      985      1111      12      3824      4635
ulmus@ulmus:~/hw4$
```

图 4-8 创建 kind 三节点集群（验证项：本地搭建 K8s 集群）

说明：集群创建耗时 46.0 s；刚创建时节点为 NotReady，CNI 就绪后变为 Ready。创建前后虚拟机已用内存从 985 MB 增至 1461 MB。

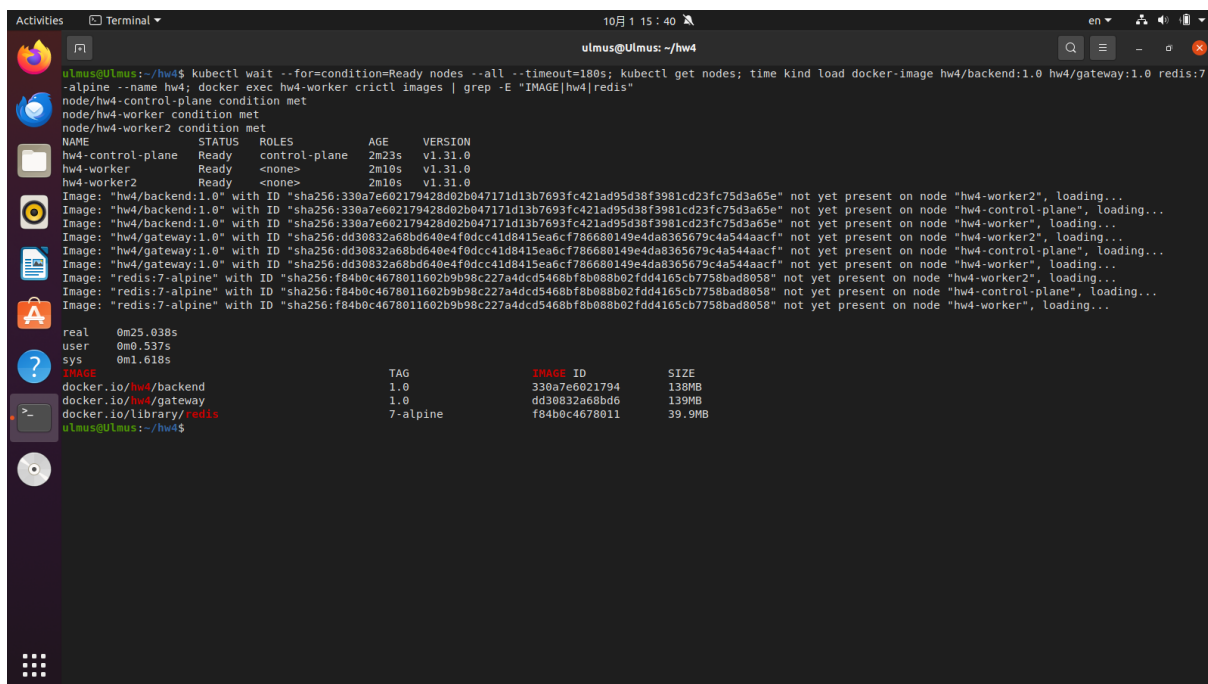


图 4-9 节点就绪并导入本地镜像

说明：kind 的节点是容器，看不到虚拟机里的 Docker 镜像，需要 kind load 把镜像导入每个节点的 containerd，耗时 25 s。

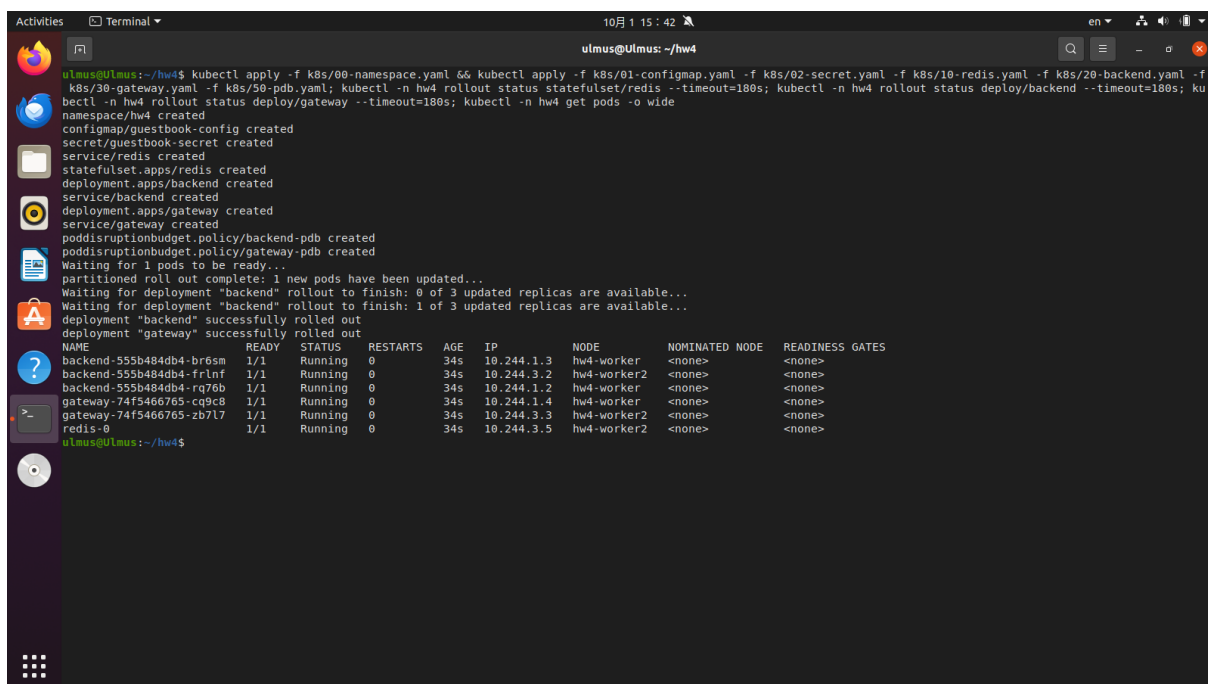


图 4-10 提交资源清单并等待滚动完成（验证项：Deployment、Service 提交）

说明：3 个 backend 与 2 个 gateway Pod 被分散调度到 hw4-worker 与 hw4-worker2 两个节点。

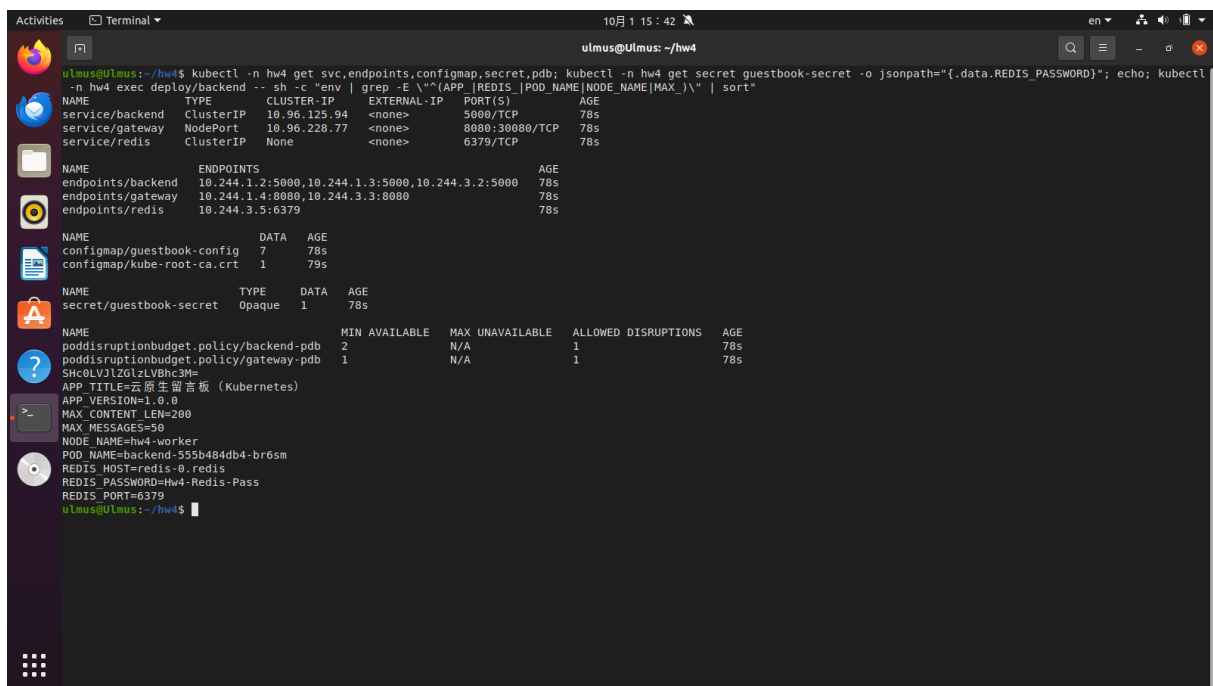


图 4-11 Service、Endpoints 与配置注入（验证项：ConfigMap / Secret 注入）

说明：backend 的 Endpoints 列出 3 个 Pod IP；Secret 在 API 中以 base64 存储；进入 Pod 可见 ConfigMap 中的 APP_TITLE、REDIS_HOST 与 Secret 中的 REDIS_PASSWORD 均已成为环境变量，POD_NAME、NODE_NAME 来自 Downward API。镜像中没有任何一项配置。

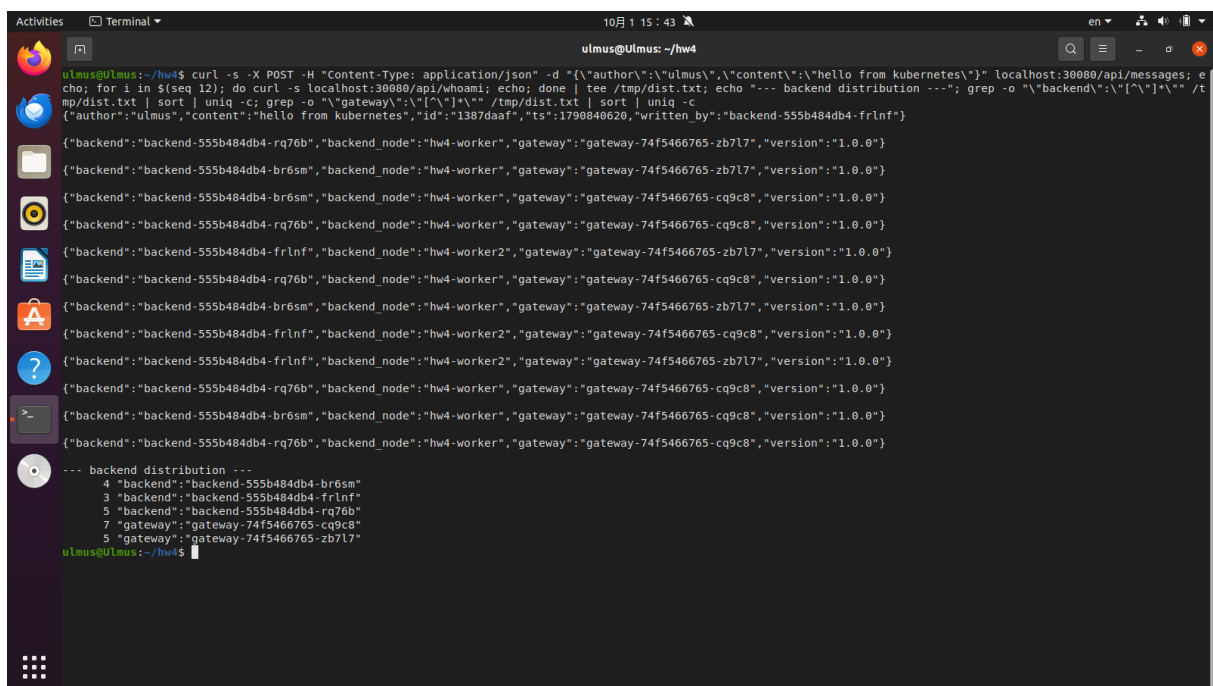


图 4-12 通过 NodePort 访问并统计请求分发（验证项：部署成功可访问）

说明：12 次请求中，3 个 backend 分别处理 4、3、5 次，2 个 gateway 分别处理 7、5 次，且跨越两个节点。

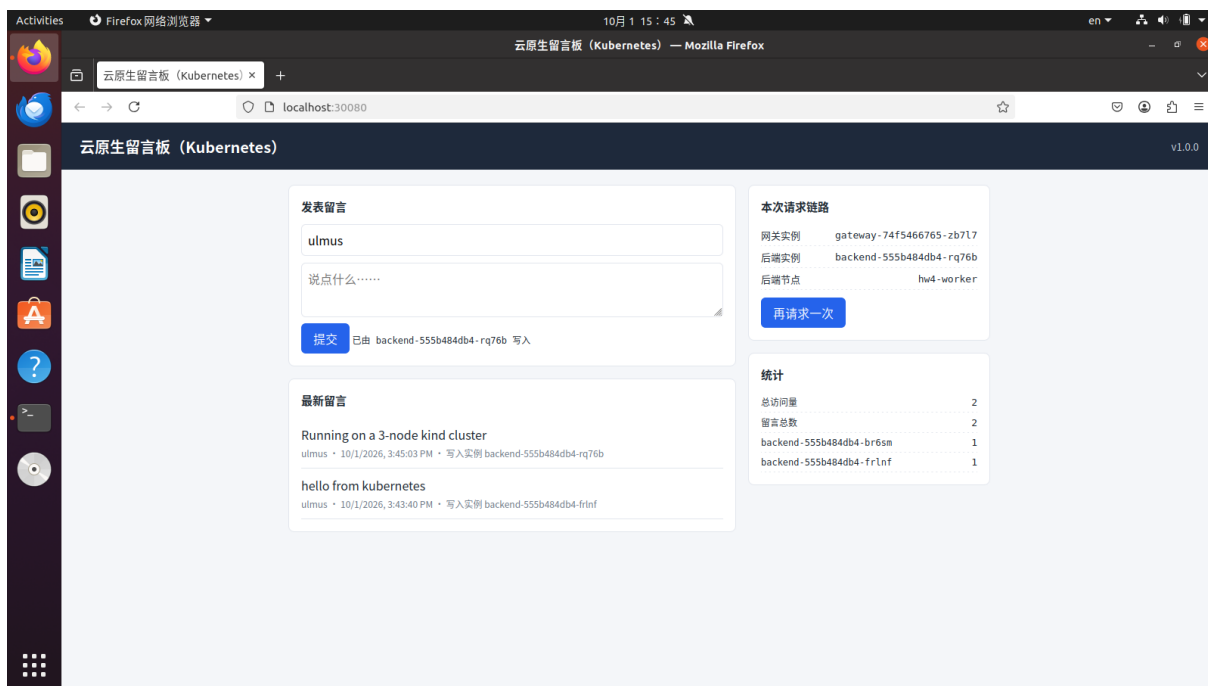


图 4-13 浏览器访问 K8s 部署的留言板

说明：标题来自 K8s 的 ConfigMap，与 compose 版本不同；请求链路显示 Pod 名与所在节点。

4.3 自愈验证

按“原始状态 → 故障注入 → 恢复结果”成组记录。故障注入方式为手动删除一个 backend Pod，同时右上窗格持续访问 /api/whoami。

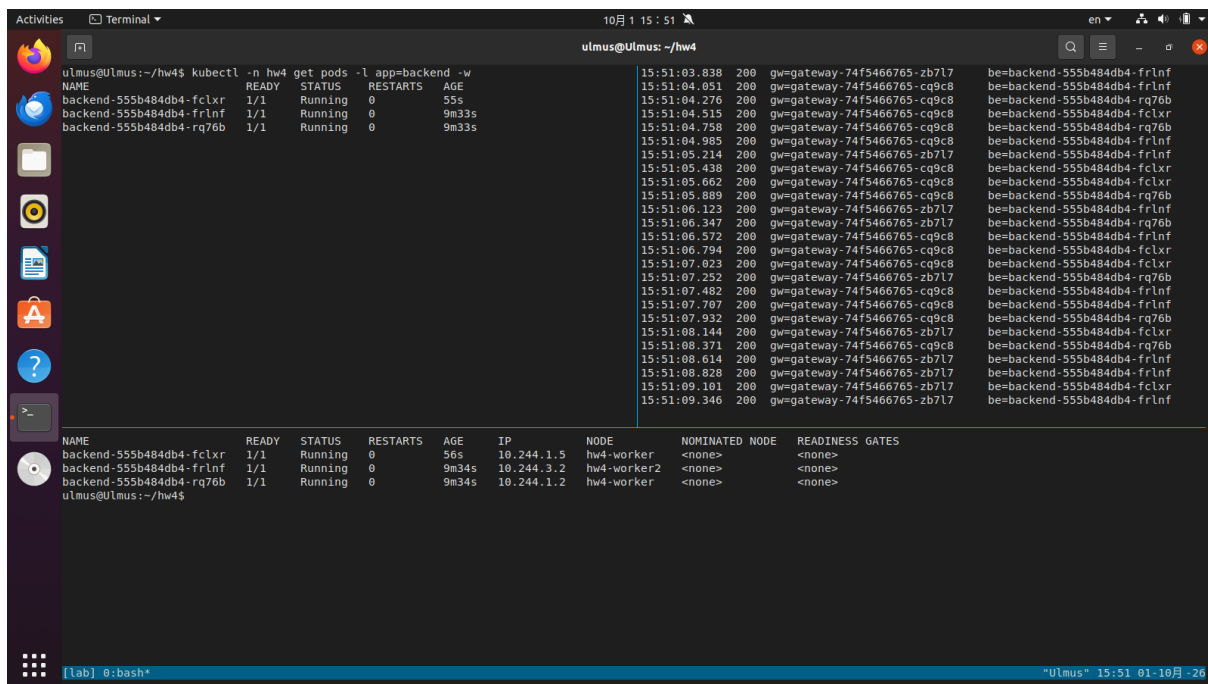


图 4-14 原始状态：3 个 backend Pod 运行，持续探测开始

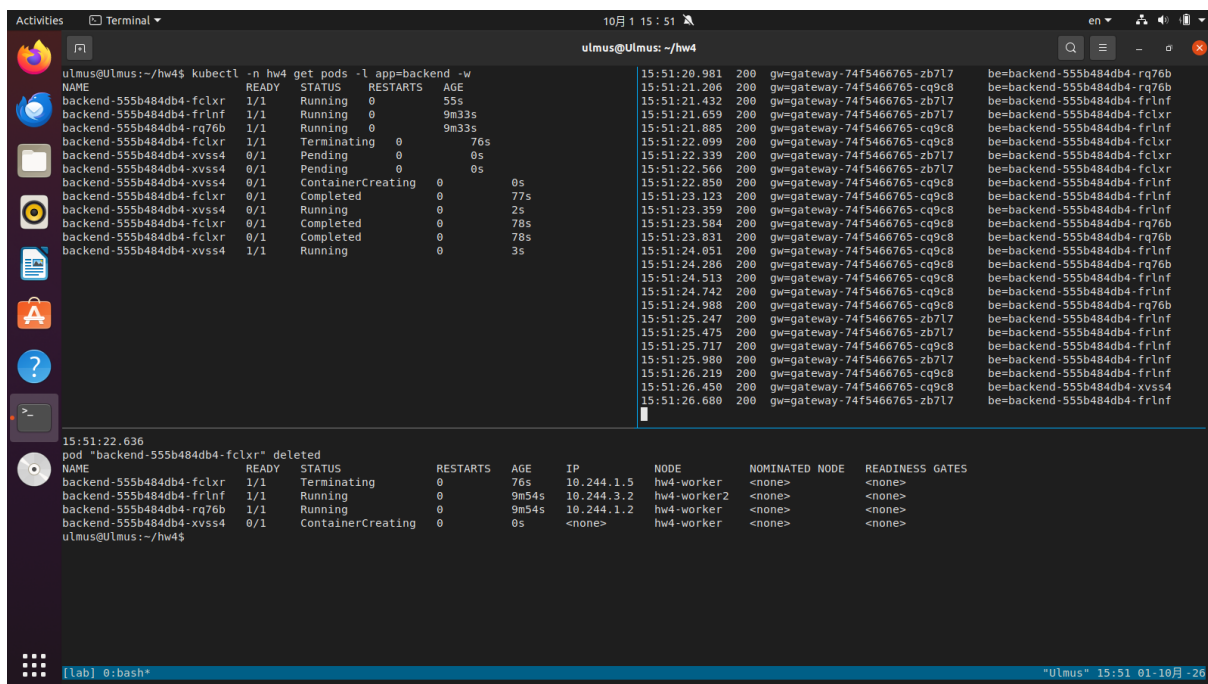


图 4-15 故障注入：删除 backend-555b484db4-fclxr（验证项：Pod 自动重建）

说明：15:51:22.6 发出删除；ReplicaSet 立即创建替代 Pod xvss4（Pending → ContainerCreating → Running），被删 Pod 进入 Terminating。右侧探测中被删 Pod 在删除后不再出现，而请求全部返回 200。

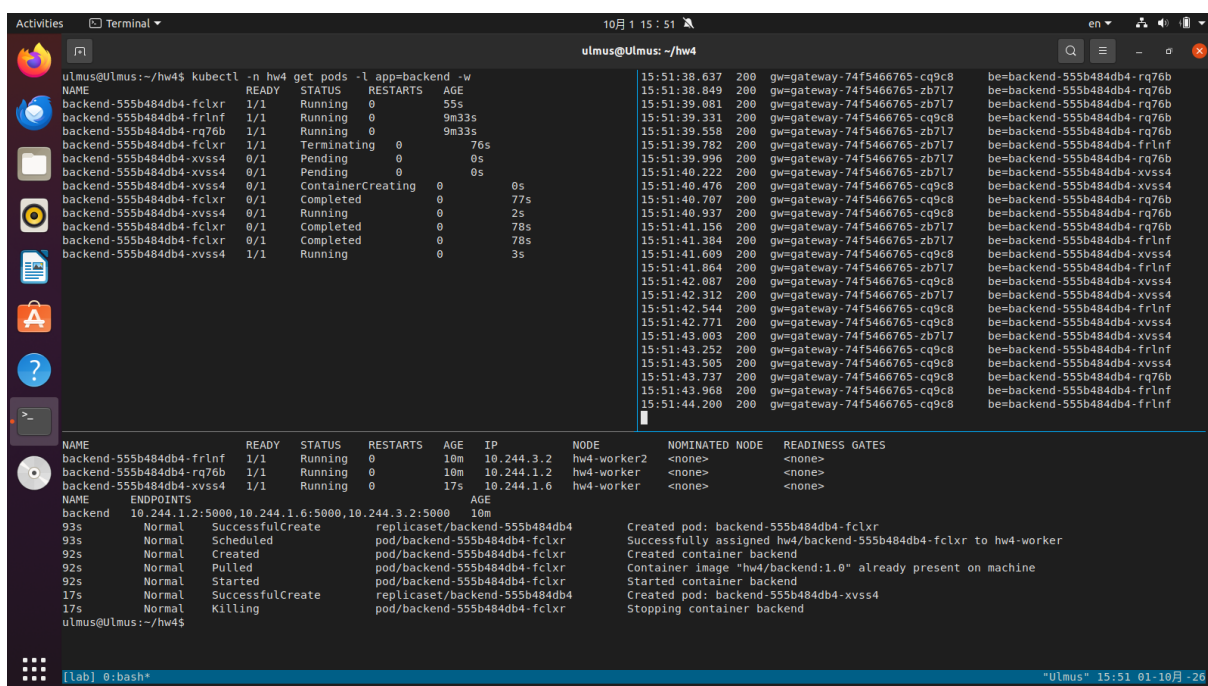


图 4-16 恢复结果：副本数回到 3，Endpoints 更新

说明：事件记录显示 Killing 与 SuccessfulCreate 在同一秒发生；新 Pod 约 3 s 后就绪并出现在 Endpoints 中，15:51:26.4 起开始接收请求。

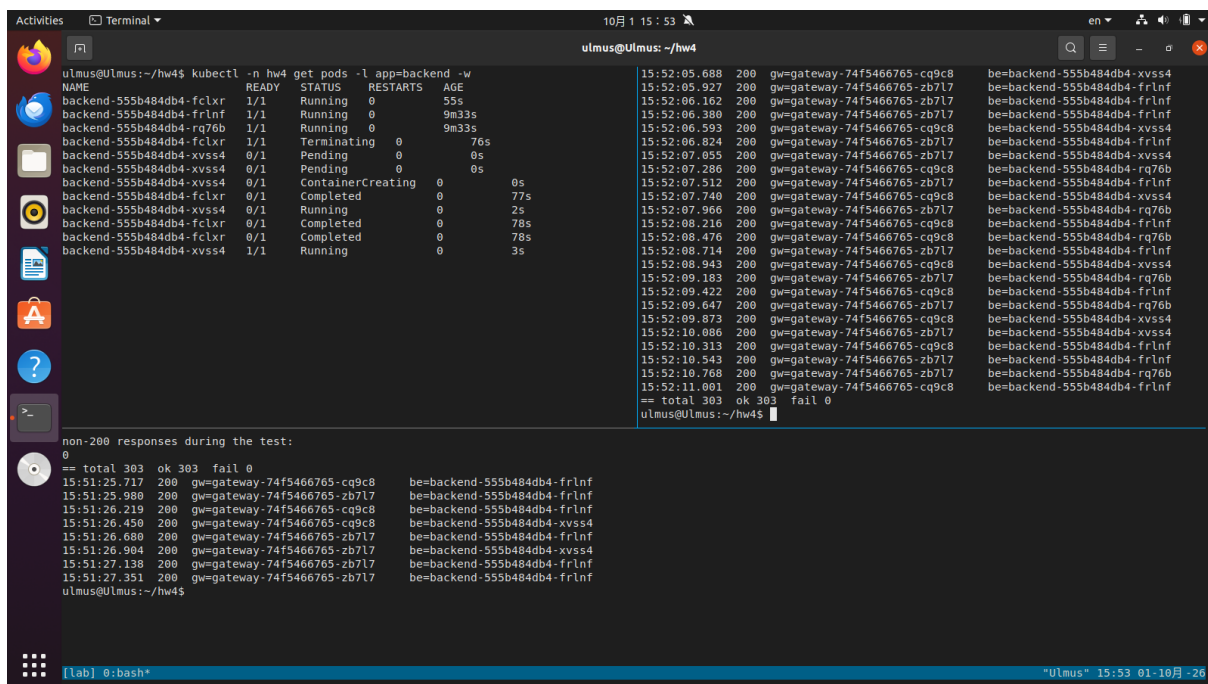


图 4-17 删除 Pod 期间的持续访问统计（验证项：服务不中断）

说明：整个过程共 303 次请求，非 200 响应为 0，新 Pod xvss4 很快参与到负载中。

4.4 弹性伸缩

先做手动扩缩容，记录 Pod 数量和资源变化；再部署 HPA，用压测触发自动扩缩容。资源数据由 metrics-server 提供。

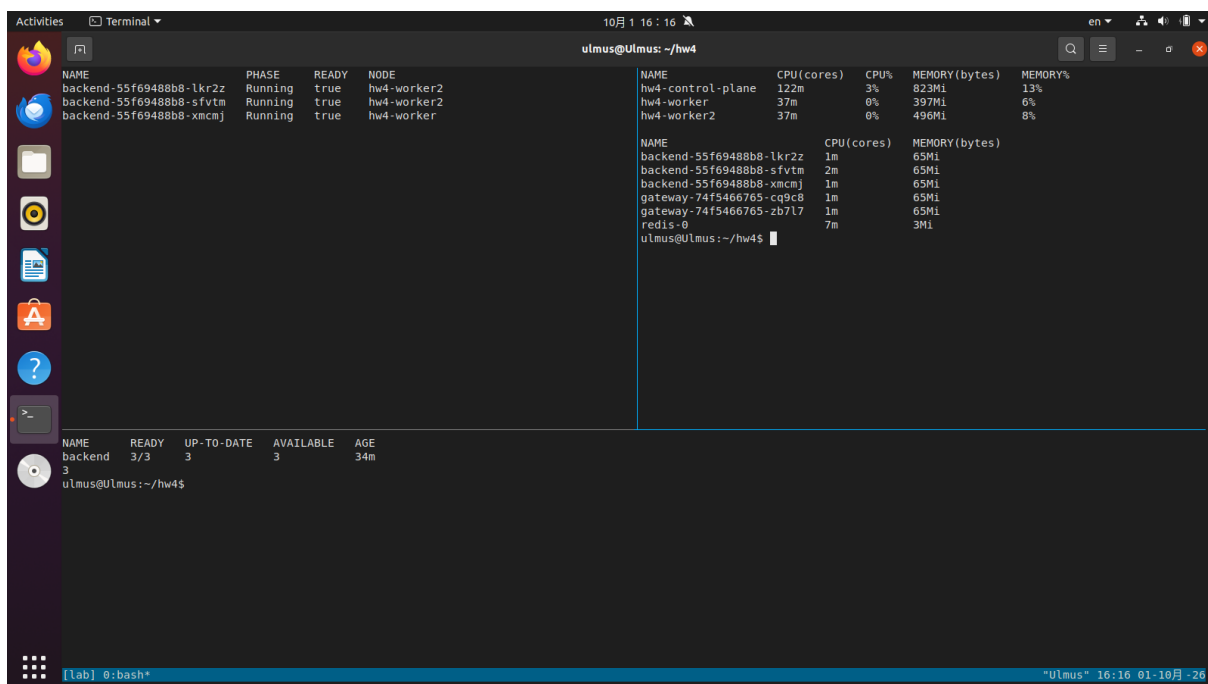


图 4-18 扩容前：3 个 backend Pod 及节点 / Pod 资源占用

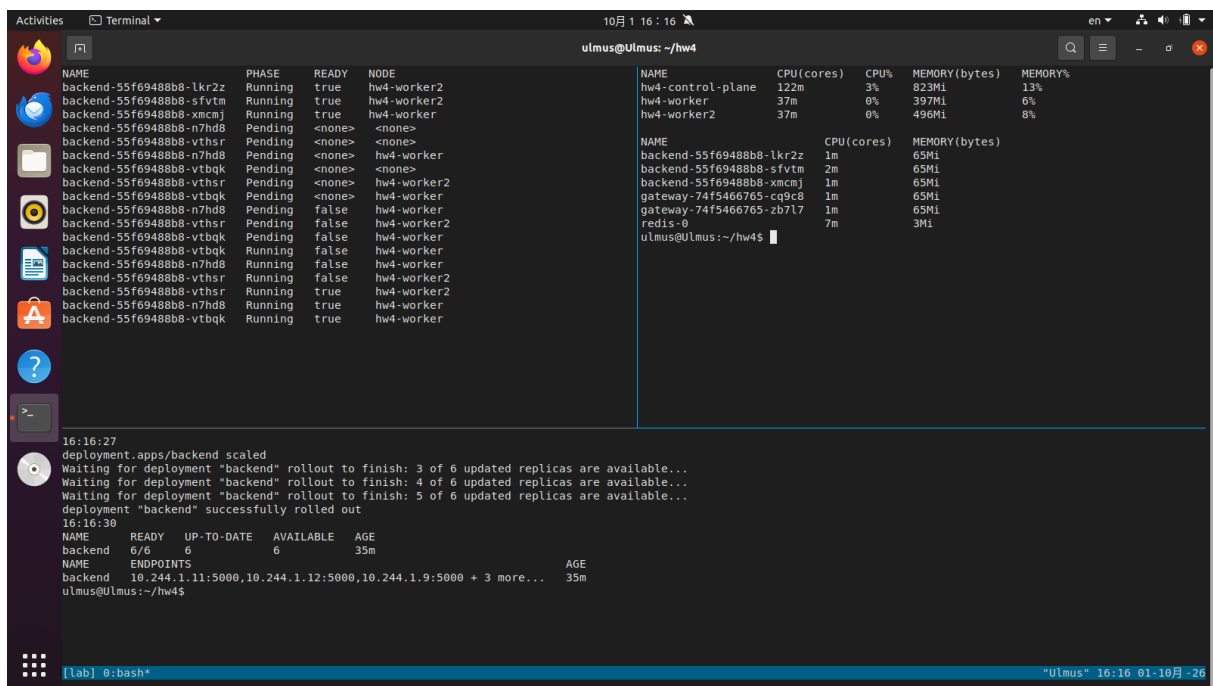


图 4-19 手动扩容到 6 副本

说明：16:16:27 执行扩容，16:16:30 rollout 完成，从 3 到 6 副本仅用约 3 s。

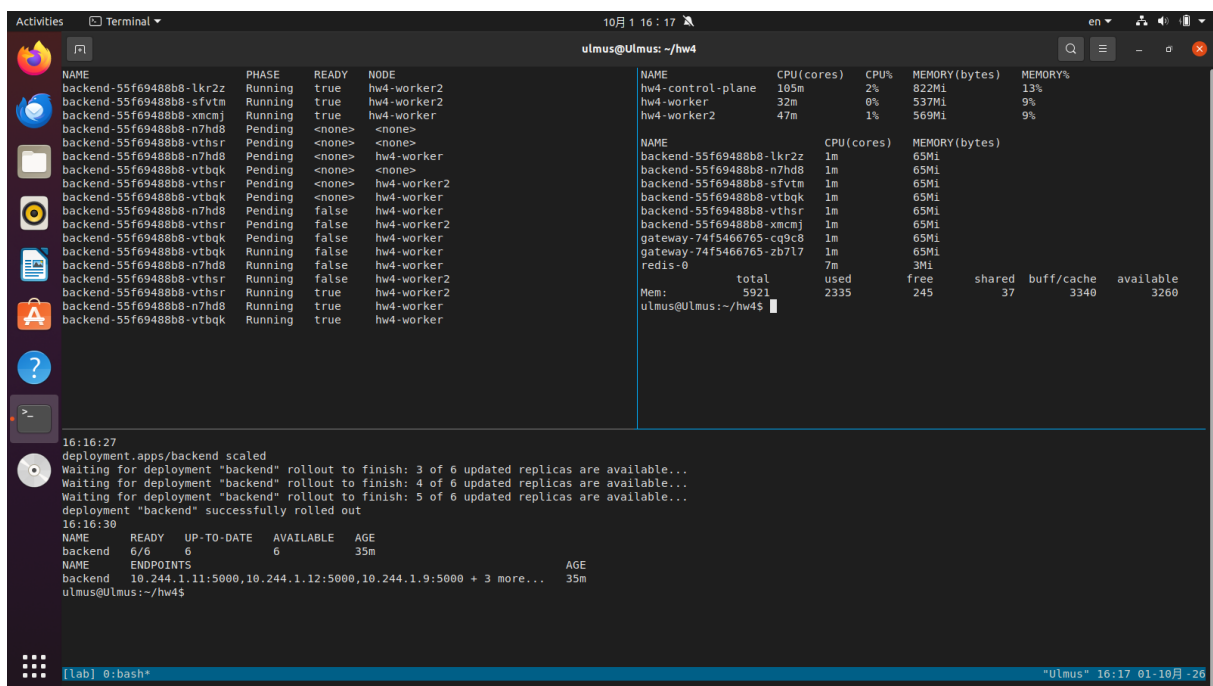


图 4-20 扩容后的资源占用（验证项：扩缩容前后资源变化）

表 4-2 手动扩容前后对比

指标	扩容前（3 副本）	扩容后（6 副本）	变化
backend Pod 数	3	6	+3
每个 backend Pod 内存	65 Mi	65 Mi	不变
hw4-worker 节点内存	397 Mi	537 Mi	+140 Mi
hw4-worker2 节点内存	496 Mi	569 Mi	+73 Mi

指标	扩容前（3 副本）	扩容后（6 副本）	变化
空闲时 Pod CPU	1~2 m	1 m	空载几乎为 0

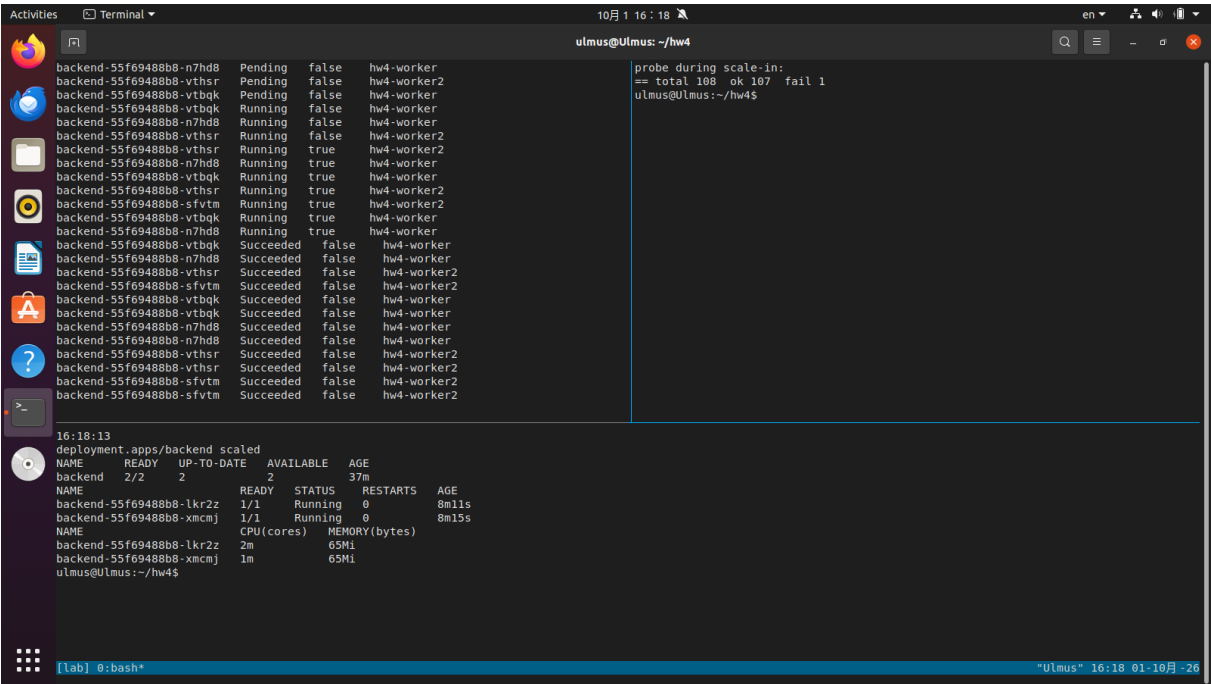


图 4-21 缩容到 2 副本：出现 1 次 502

说明：108 次请求中有 1 次失败（16:18:14.191，gateway 返回 502）。原因与解决方法见第六章问题 6。

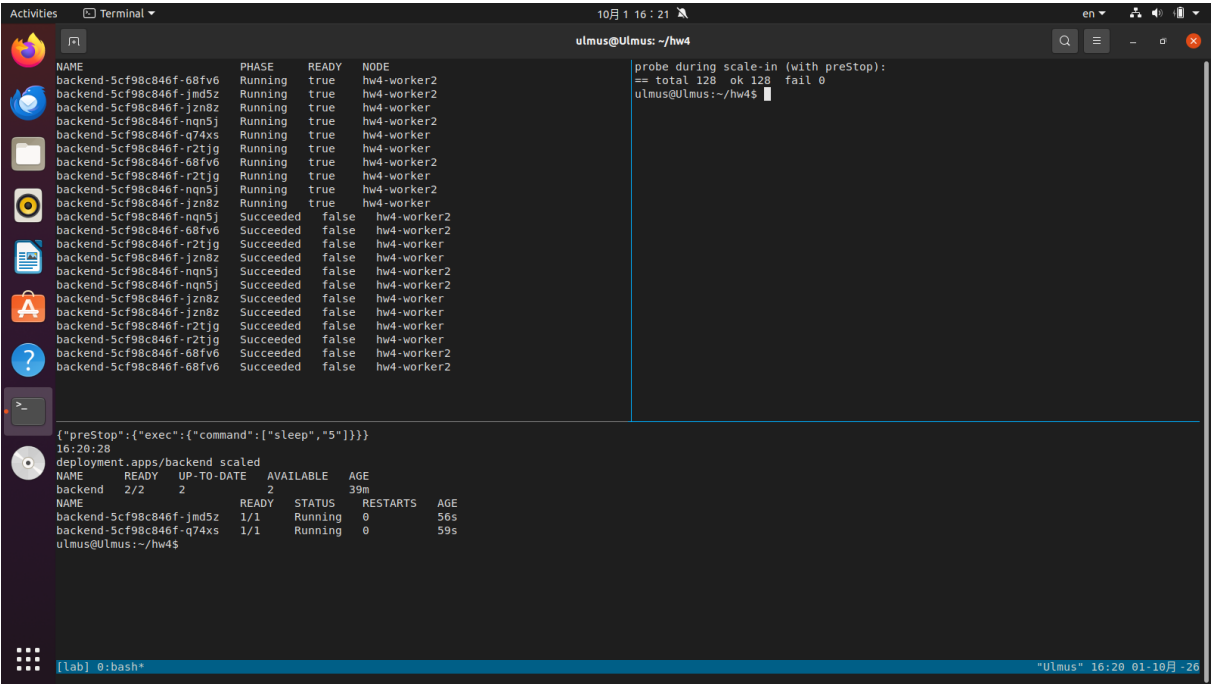


图 4-22 增加 preStop 后再次从 6 缩容到 2

说明：Deployment 中可见 preStop 配置为 sleep 5；同样的缩容过程 128 次请求全部成功。

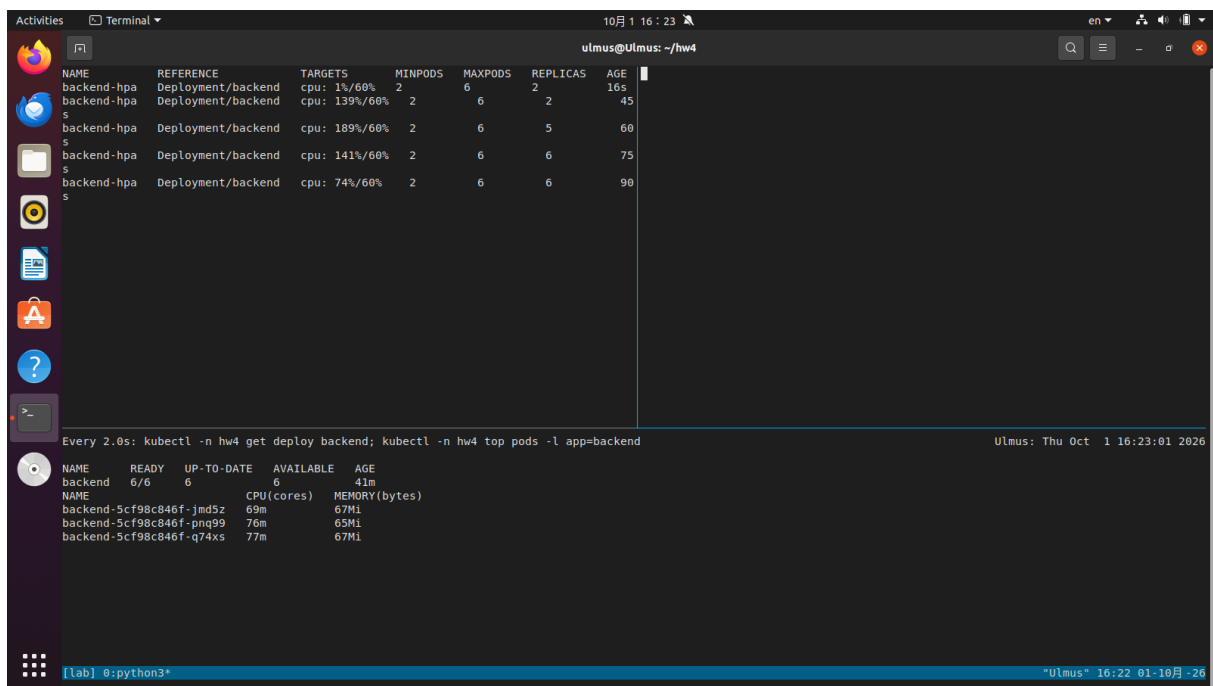


图 4-23 HPA 自动扩容（验证项：弹性伸缩）

说明：以并发 40 持续压测，backend 平均 CPU 利用率达到 139%、189%（相对 request 100m），HPA 在约 30 s 内把副本从 2 扩到 5、再到 6（上限）。

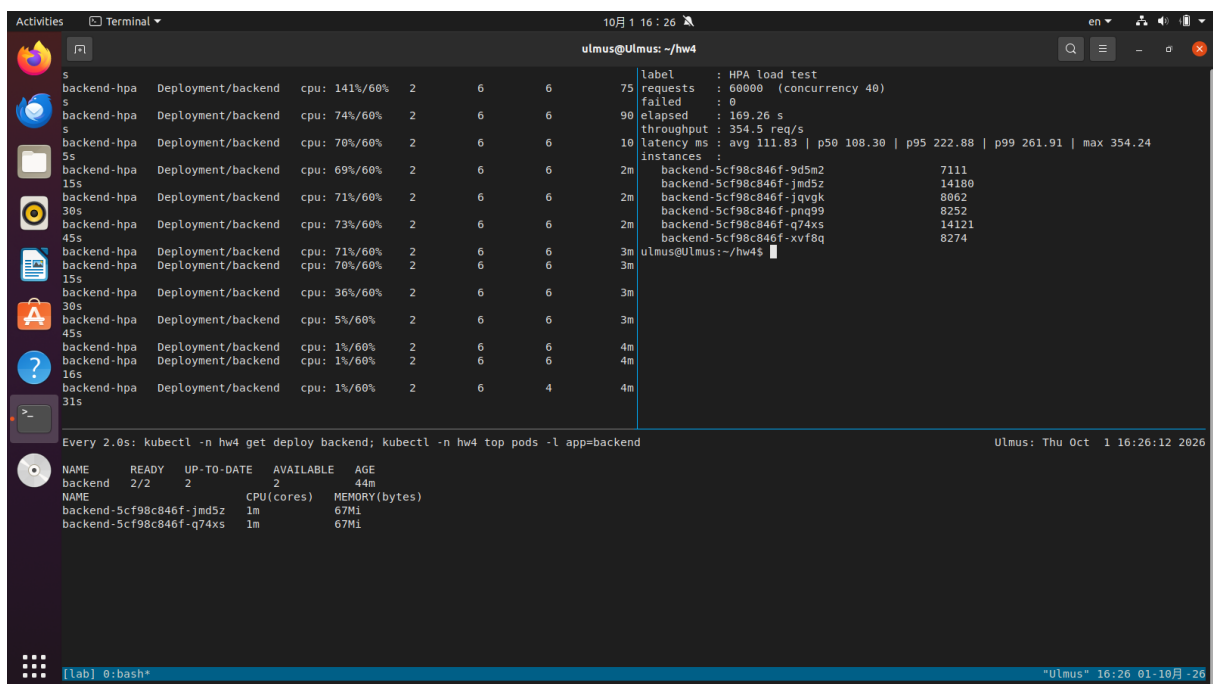


图 4-24 压测结束后 HPA 自动缩容

说明：右上为压测结果：60000 次请求失败 0 次，6 个副本都分担了流量。负载消失后 CPU 降到 1%，经过 60 s 稳定窗口，副本数按 6 → 4 → 2 回落到最小值。

4.5 进阶资源与高可用验证

4.5.1 StatefulSet 与数据持久化

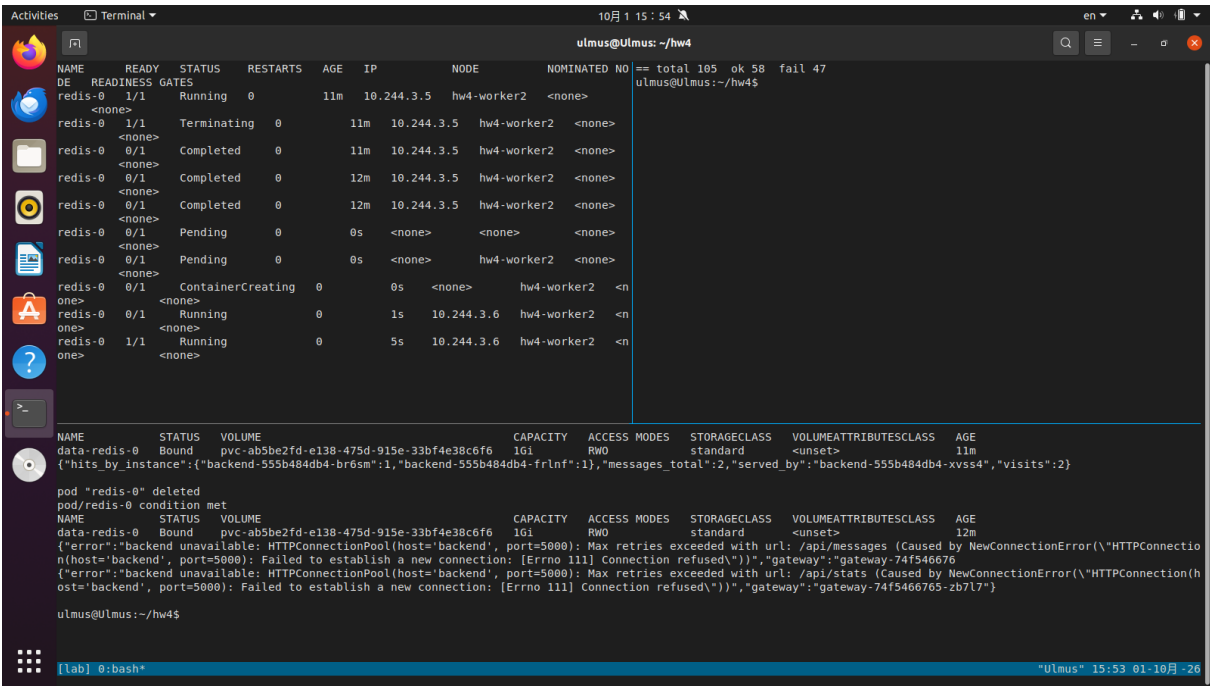


图 4-25 删除 redis-0: Pod 以同名重建并挂载原 PVC

说明：StatefulSet 重建的 Pod 仍叫 redis-0，PVC data-redis-0 的卷名不变。但右上探测显示 105 次请求失败 47 次，连不依赖 Redis 的 /api/whoami 也失败了，这是 v1.0 的设计缺陷（第六章问题 4）。

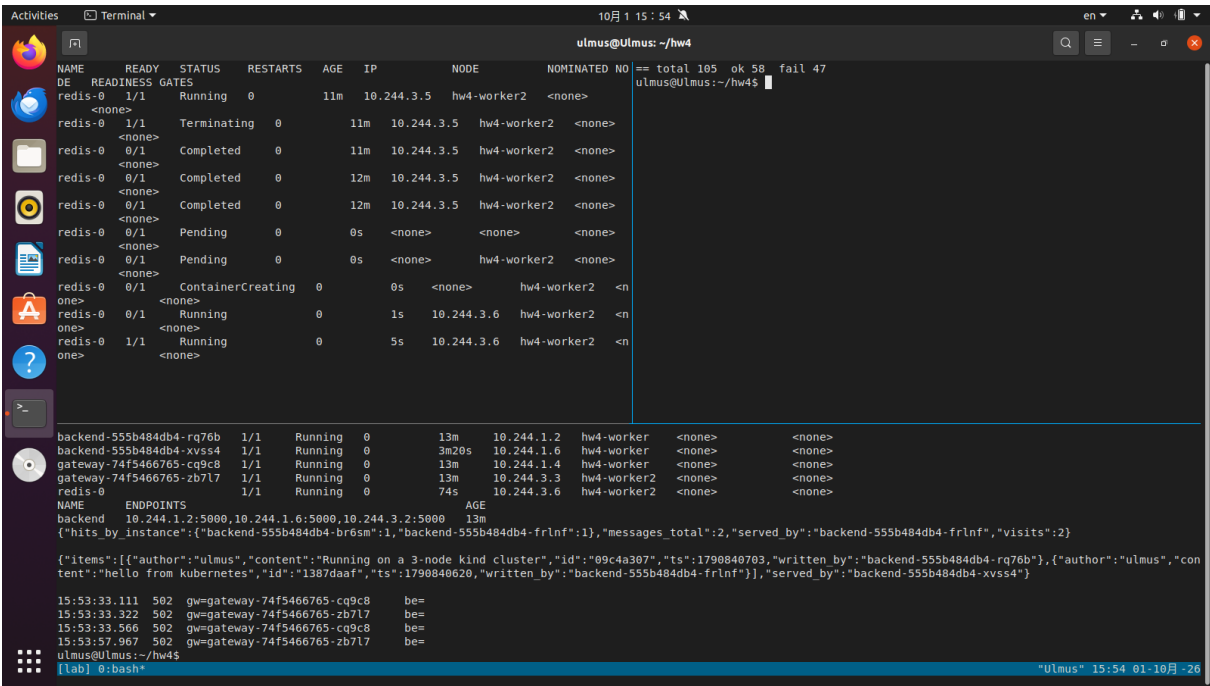


图 4-26 Redis 恢复后数据完整

说明：留言总数与两条留言内容、各实例命中统计均与删除前一致；失败集中在 15:53:33~15:53:57 约 25 s。

4.5.2 滚动更新

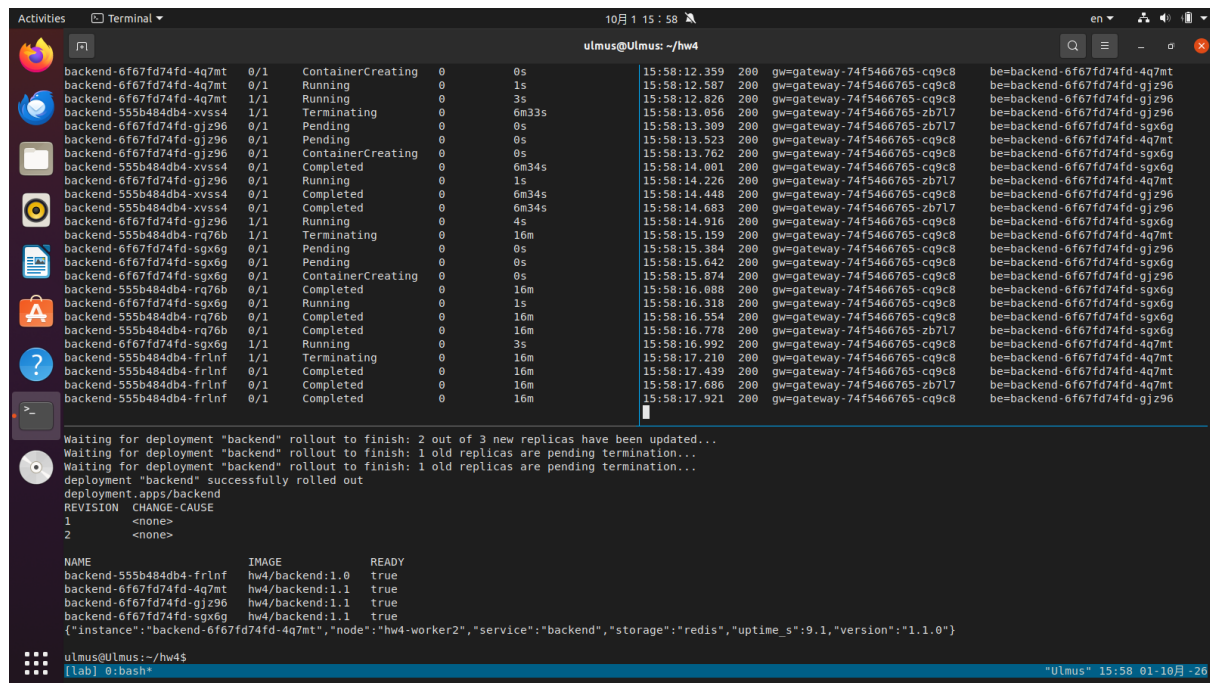


图 4-27 backend 从 1.0 滚动更新到 1.1

说明：maxSurge=1、maxUnavailable=0，新 Pod 就绪后才终止旧 Pod；rollout history 记录了两个版本，新 Pod 返回 version 1.1.0。

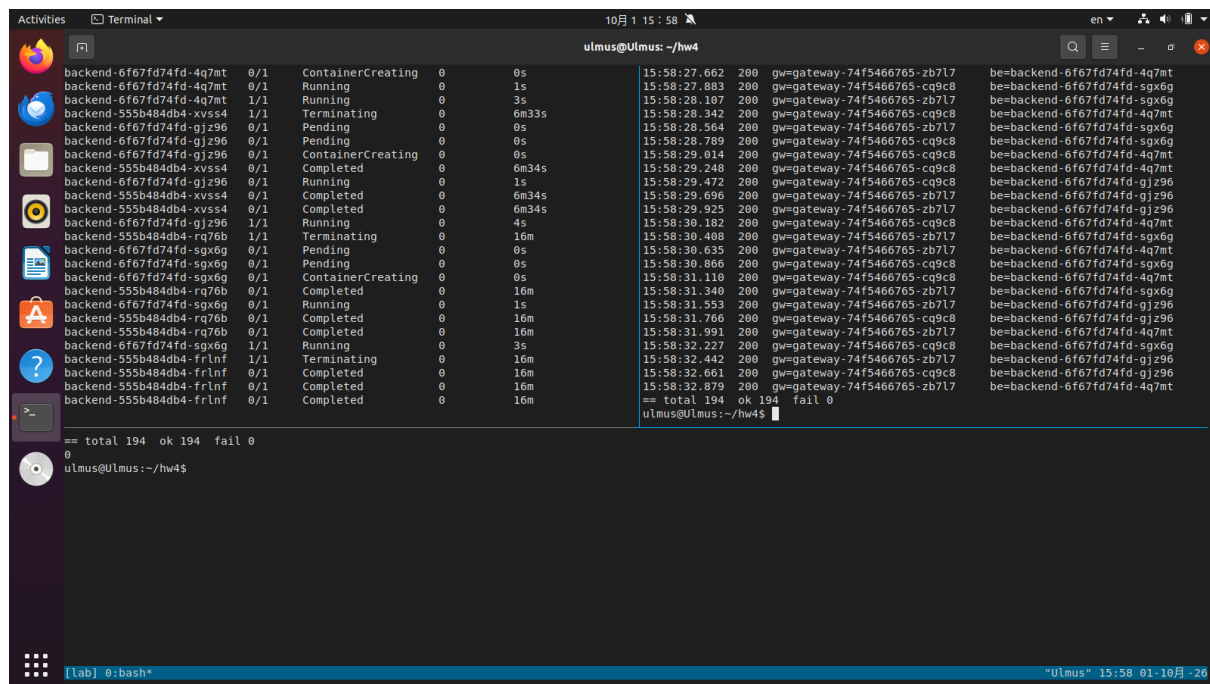


图 4-28 滚动更新期间服务可用性

说明：194 次请求全部成功。

4.5.3 PodDisruptionBudget 与节点排空

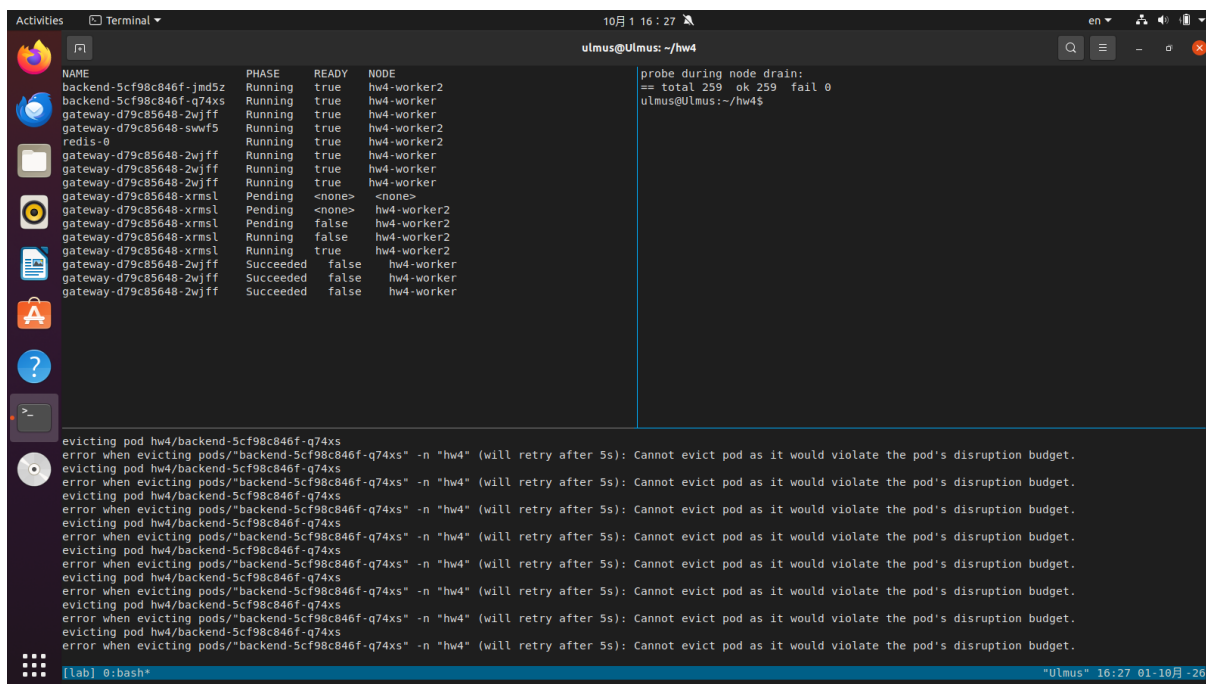


图 4-29 排空 hw4-worker: backend 被 PDB 阻止驱逐

说明: gateway 被正常迁移, 但 backend 驱逐一直报“would violate the pod's disruption budget”, 最终超时(第六章问题 7)。服务本身 259 次请求全部成功。

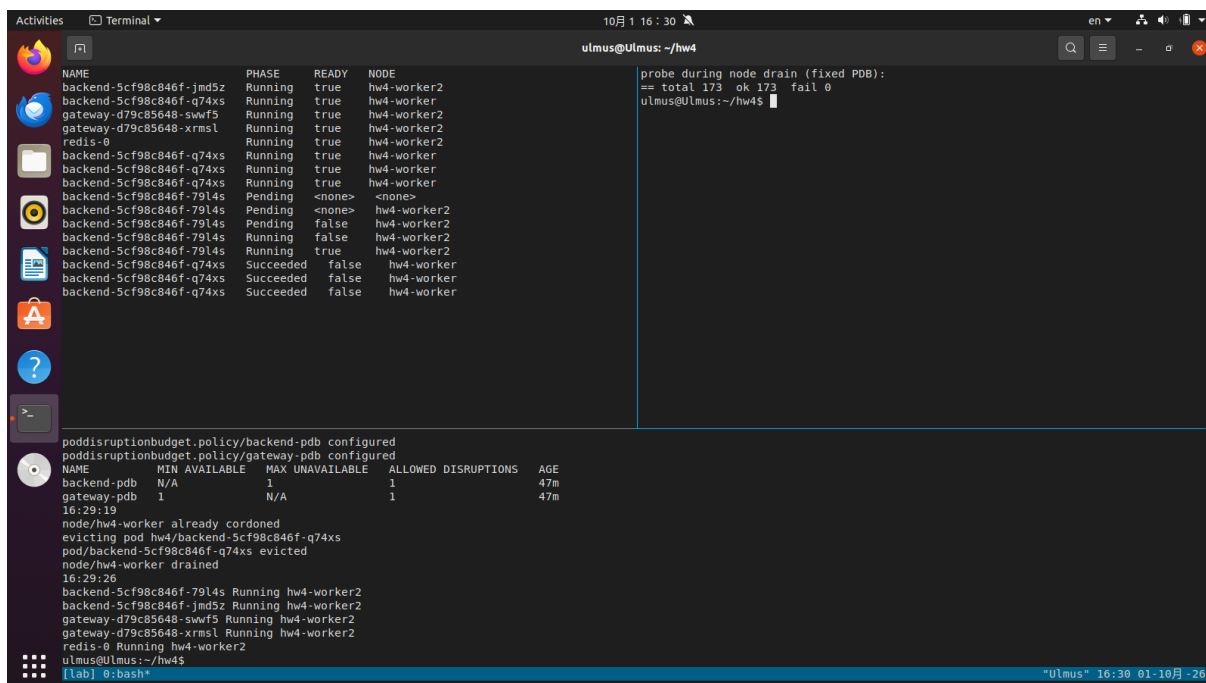


图 4-30 修改 PDB 后成功排空节点

说明：PDB 改为 maxUnavailable=1 后，backend 先在 worker2 上新建副本再驱逐旧副本，7 s 完成排空，173 次请求全部成功。

4.5.4 节点故障（非计划中断）

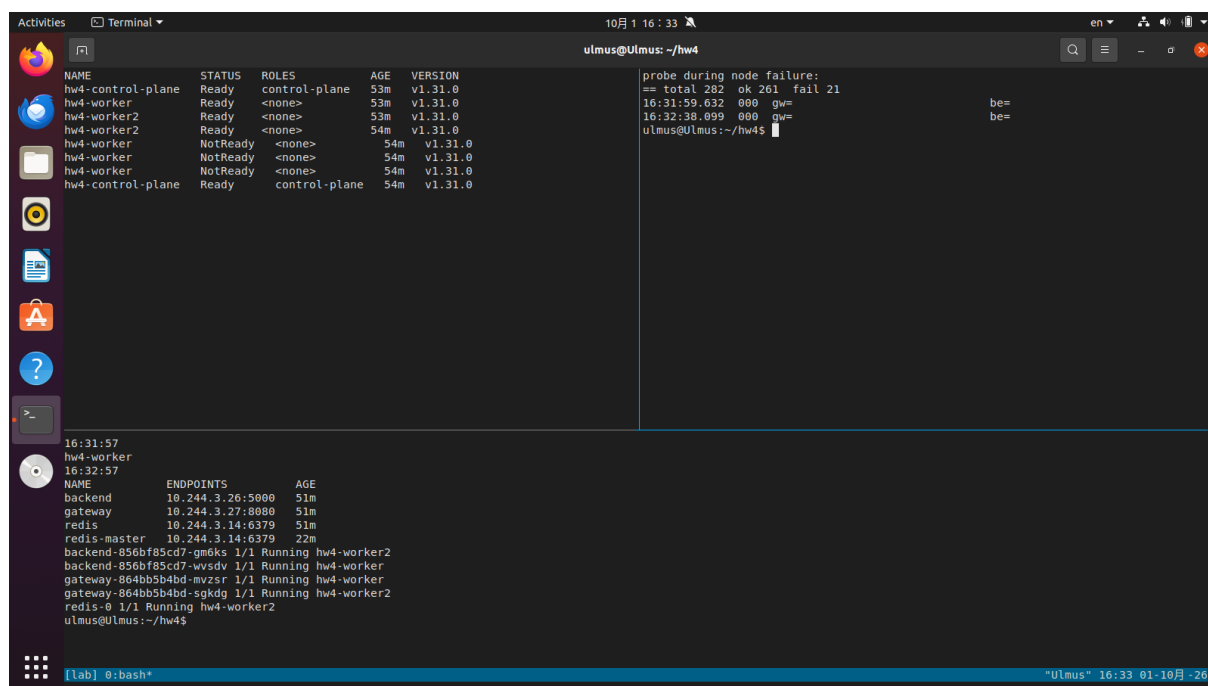


图 4-31 直接停止 hw4-worker 节点

说明：16:31:57 用 docker stop 模拟节点宕机。约 40 s 后节点被标记为 NotReady，其上的 Pod 被移出 Endpoints；在此之前转发到该节点的请求超时，282 次请求中失败 21 次，集中在 16:31:59~16:32:38。之后服务完全由 worker2 承担。

4.5.5 Ingress

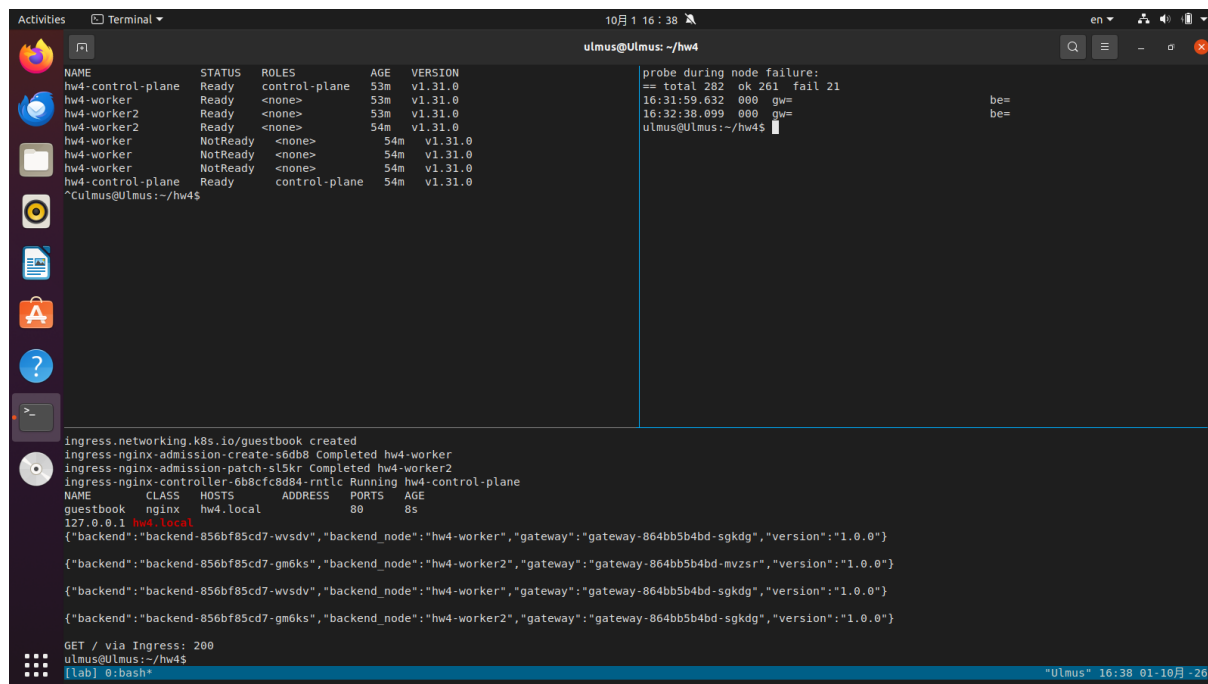


图 4-32 通过 Ingress 域名 hw4.local 访问

说明：ingress-nginx 控制器运行在带 ingress-ready 标签的控制面节点上；请求经 Ingress 后同样被分发到不同节点的 gateway 与 backend。

4.5.6 CI 流水线

仓库中的 `.github/workflows/ci.yml` 在每次推送到 `main` 分支时自动执行：pyflakes 静态检查 → 构建两个镜像 → 用 `docker compose` 启动并对页面、留言写入与读取做冒烟测试 → 用 `kubeconform` 校验全部 K8s 清单 → 把镜像推送到 GitHub Container Registry。整个流程约 1 分钟，任何一步失败都会阻止镜像发布。

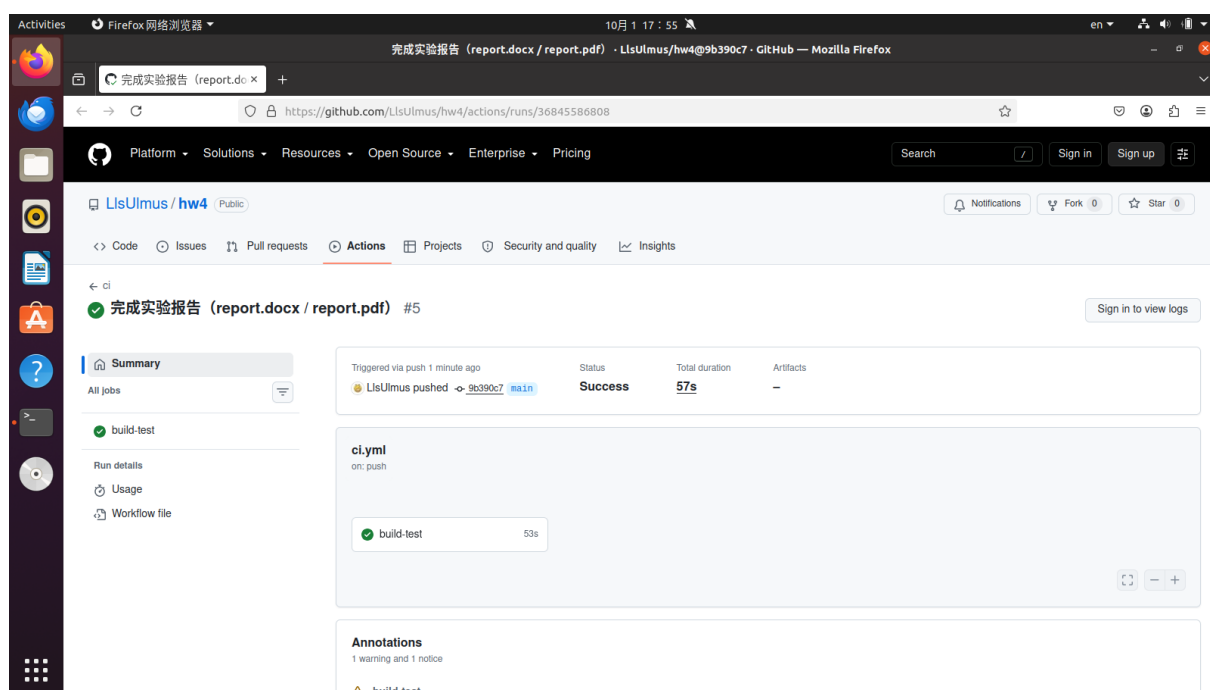


图 4-33 GitHub Actions 中 CI 流水线运行成功

说明：本次运行由提交 9b390c7 触发，build-test 作业耗时 53 s，状态为 Success。

4.6 Docker 与 K8s 的差异化比较

同一应用在 `docker compose`（4.1 节）与 K8s（4.2~4.5 节）两条路径下各部署了一遍。为了让比较有依据，在 Docker 侧补做了两种故障注入：容器主进程退出与容器被删除。

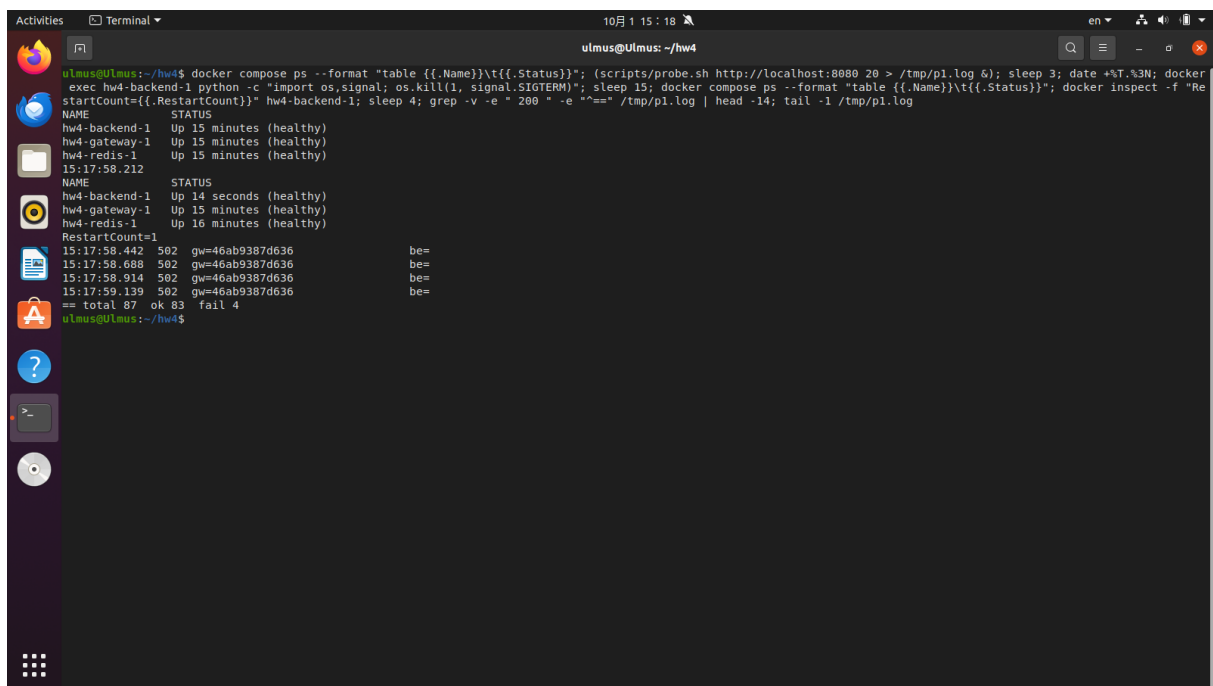


图 4-34 Docker：容器主进程退出后由重启策略拉起

说明：向 backend 的 PID 1 发送 SIGTERM，restart: unless-stopped 使容器在原地重启（RestartCount=1）。由于只有 1 个副本且没有就绪门控，期间 4 次请求返回 502。

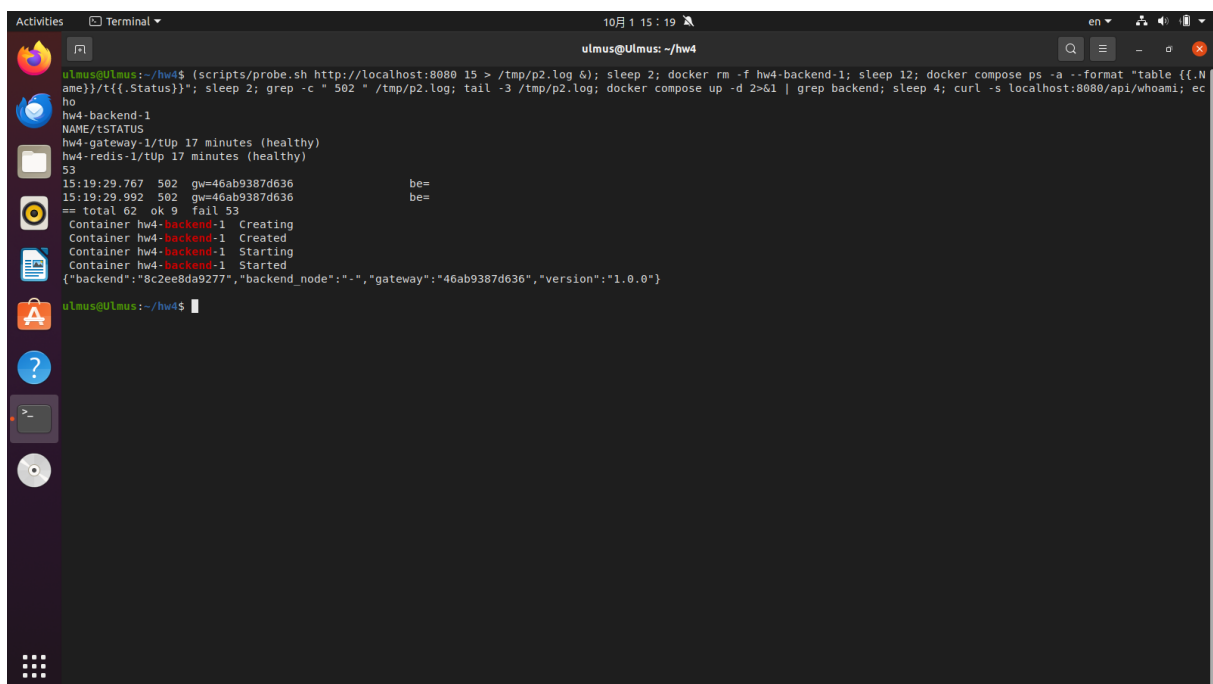


图 4-35 Docker：容器被删除后不会自动重建

说明：docker rm -f 后，compose 不会自行补回容器，62 次请求失败 53 次，直到手动执行 docker compose up -d 才恢复。

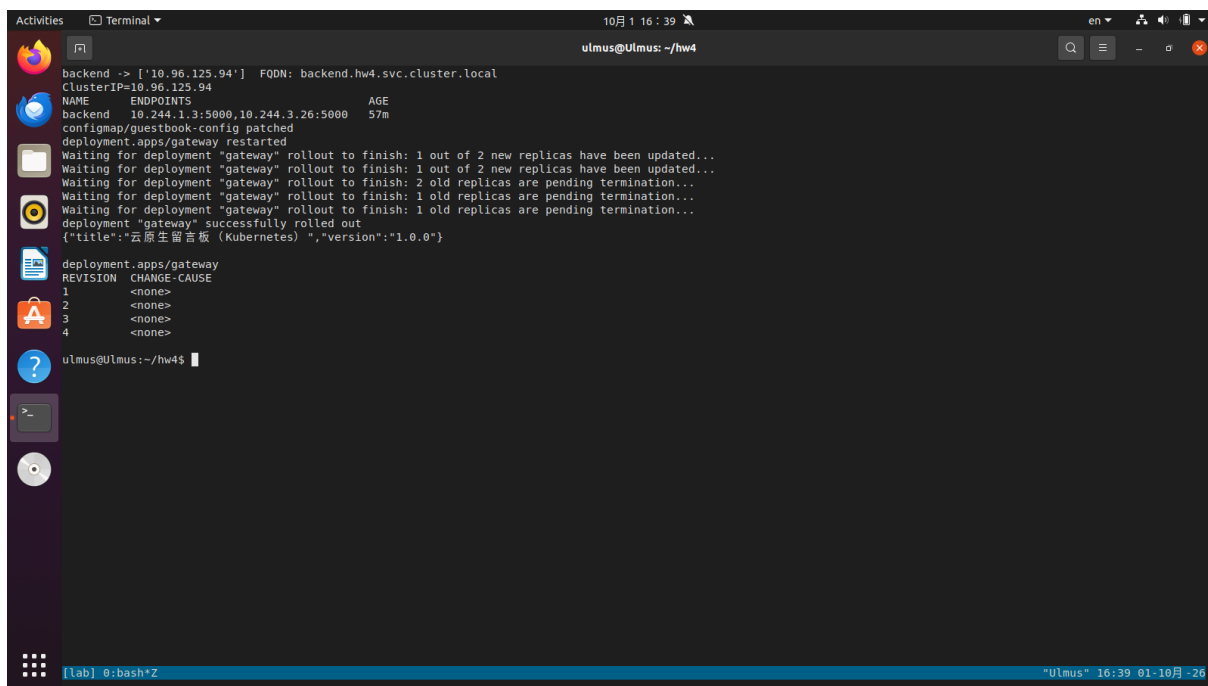


图 4-36 K8s: 服务发现与配置变更

说明：在 K8s 中，服务名 backend 解析为单一的 ClusterIP（10.96.125.94），由 kube-proxy 把流量分到 Endpoints 中的 Pod；Docker 中服务名直接解析为多个容器 IP（图 4-5）。修改 ConfigMap 后需要 rollout restart 才能生效。

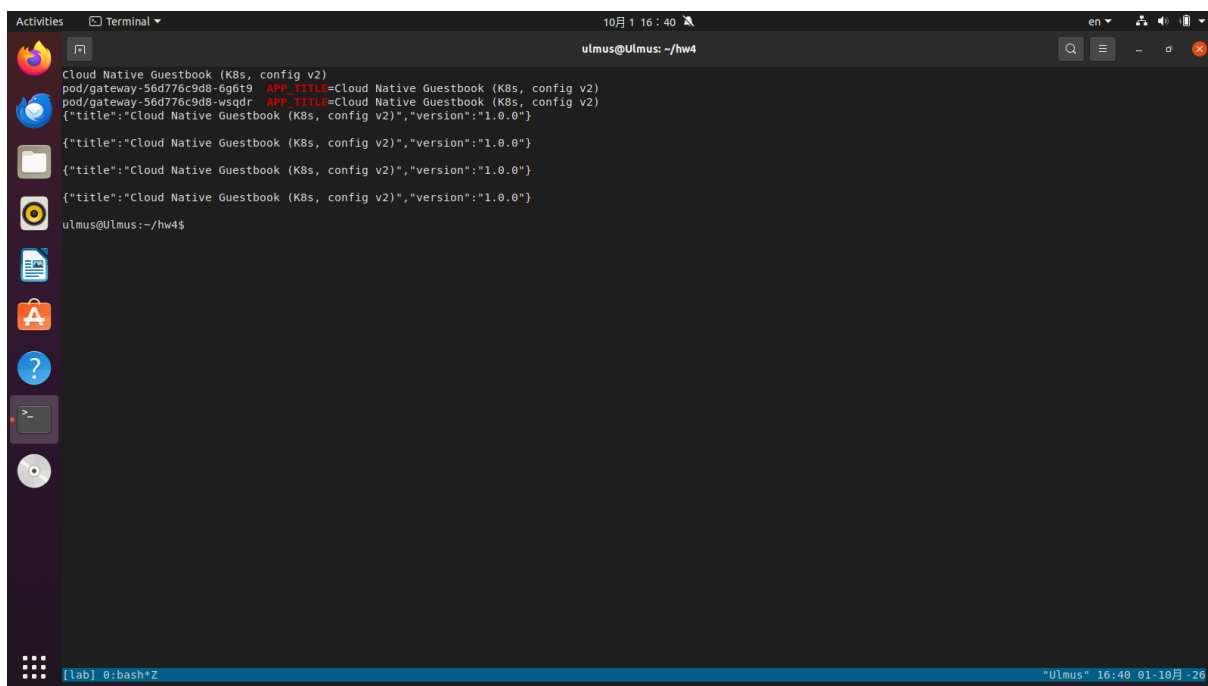


图 4-37 K8s: 配置变更在新 Pod 中生效

说明：图 4-36 中 rollout 刚完成时那次请求仍返回旧标题，因为请求落到了处于 preStop 等待期的旧 Pod；稍后所有请求均返回新标题。

表 4-3 Docker Compose 与 Kubernetes 的对比（基于本次实测）

维度	Docker / Compose	Kubernetes	实测依据
多副本管理	--scale 临时指定，down/up 后不保留	replicas 写在声明里，控制器持续对齐	图 4-6、图 4-19
故障恢复	仅进程退出时原地重启；容器被删不补回	Pod 被删立即重建并重新调度；节点宕机后摘流	图 4-34/35 vs 图 4-15/31
故障期间可用性	单副本重启 4 次 502；容器删除 53 次失败	删 Pod 0 失败；滚动更新 0 失败	probe 统计
配置管理	环境变量写在 compose / .env 中，随文件分发	ConfigMap / Secret 独立对象，可单独更新并滚动	图 4-11、图 4-36
服务发现	内置 DNS，服务名解析为多个容器 IP	ClusterIP 虚拟 IP + Endpoints，DNS 记录稳定	图 4-5、图 4-36
横向扩展	手动 --scale	手动 scale 或 HPA 按指标自动伸缩	图 4-23/24
发布方式	重建容器，单副本必然中断	滚动更新、可回滚、可设最大不可用数	图 4-27/28
运维对象	单机上的容器	多节点集群；可排空节点、限制中断预算	图 4-29/30
上手成本	4 个文件约 84 行，3 步可访问	9 个清单约 325 行 + 集群与插件，约 7 步	第五章表 5-4

结合操作过程，两者解决的核心问题并不相同。Docker 解决的是“交付”问题：把应用和它的运行环境打成一个镜像，在哪里都能以同样的方式启动，并获得进程级的隔离——这一点在第一次 docker run 时就能体会到，几乎没有学习成本。Kubernetes 解决的是“运行”问题：它不关心容器怎么构建，而是持续地把集群的实际状态向声明的期望状态拉齐。删掉一个 Pod、关掉一个节点、改一次版本号，背后都是同一个控制循环在工作。代价是清单数量和概念数量成倍增加，而且像探针、PDB、preStop 这些配置一旦写得不合适，反而会制造故障（见第六章），所以 K8s 的价值要在多副本、需要持续可用、需要频繁发布的场景下才能体现出来。

五、消融对比实验

5.1 对比路径与测试方法

在现有设备条件下，选择了以下五种形态逐层叠加（图 2-1）：L0 物理机进程、L1 虚拟机进程、L2 虚拟机 + Docker、L3 虚拟机 + kind K8s、L4 虚拟机 + KVM 嵌套虚拟机 + minikube K8s。测试方法统一如下：

- **性能：**同一个 bench.py 客户端对 /api/whoami 发 2000 个请求、并发 20，每种形态测 3 次取平均。该接口经过 gateway → backend 两跳，但不访问 Redis，可以排除存储因素。
- **一致性控制：**L1~L4 的服务端均为 gunicorn -w 2；gunicorn 不支持 Windows，因此 L0 与 L1 之间额外用 waitress 做了一组同服务器对照。K8s 测试时 gateway、backend 都缩为 1 副本，与 compose 保持一致。
- **启动时间：**从发出启动命令到 /healthz 首次返回 200（K8s 为 Pod 进入 Ready）的毫秒数，各测 5 次。
- **资源：**ps RSS、docker stats、kubectl top、free、virsh domstats。
- **局限：**压测客户端与服务端在同一台机器上，会争用 CPU；虚拟机只有 4 个 vCPU，嵌套虚拟机只有 2 个，因此绝对值只在本机有意义，重点看相对变化。

5.2 启动时间

表 5-1 启动时间对比

对象	层次	实测值	说明
应用进程（gunicorn）	L1 虚拟机内	192 ms（190~196）	5 次均值
Docker 容器	L2	948 ms（895~1019）	含创建容器、网络、端口映射
kind 集群内 Pod	L3	1507 ms（827~1785）	含调度、创建、就绪探针
嵌套集群内 Pod	L4	2202 ms（1890~2621）	同上，节点为嵌套虚拟机
虚拟机操作系统引导	L1	21.5 s	systemd-analyze：内核 6.0 s + 用户态 15.5 s，不含固件
kind 三节点集群创建	L3	46.0 s	节点镜像已预拉取
嵌套 minikube 冷启动	L4	80.6 s	含嵌套虚拟机引导与 K8s 组件启动

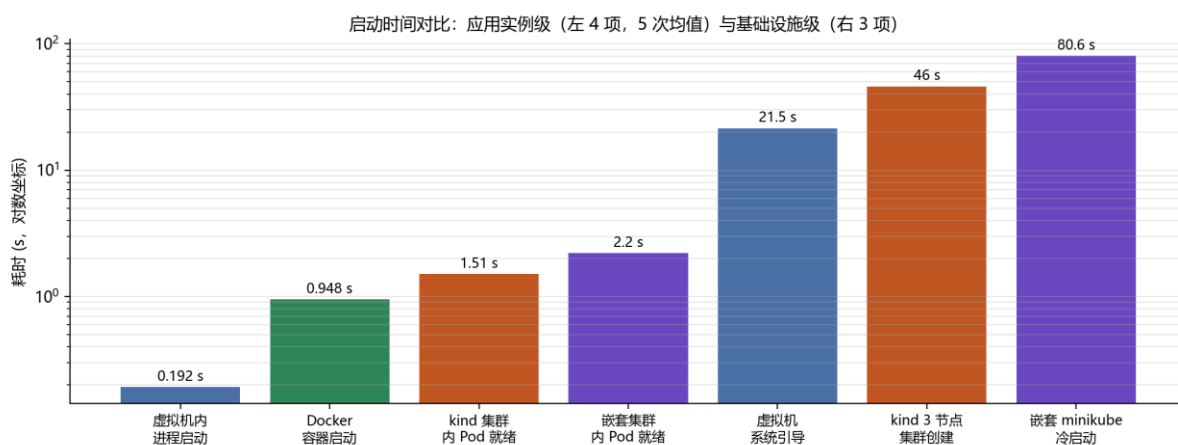


图 5-1 各层启动时间（对数坐标）

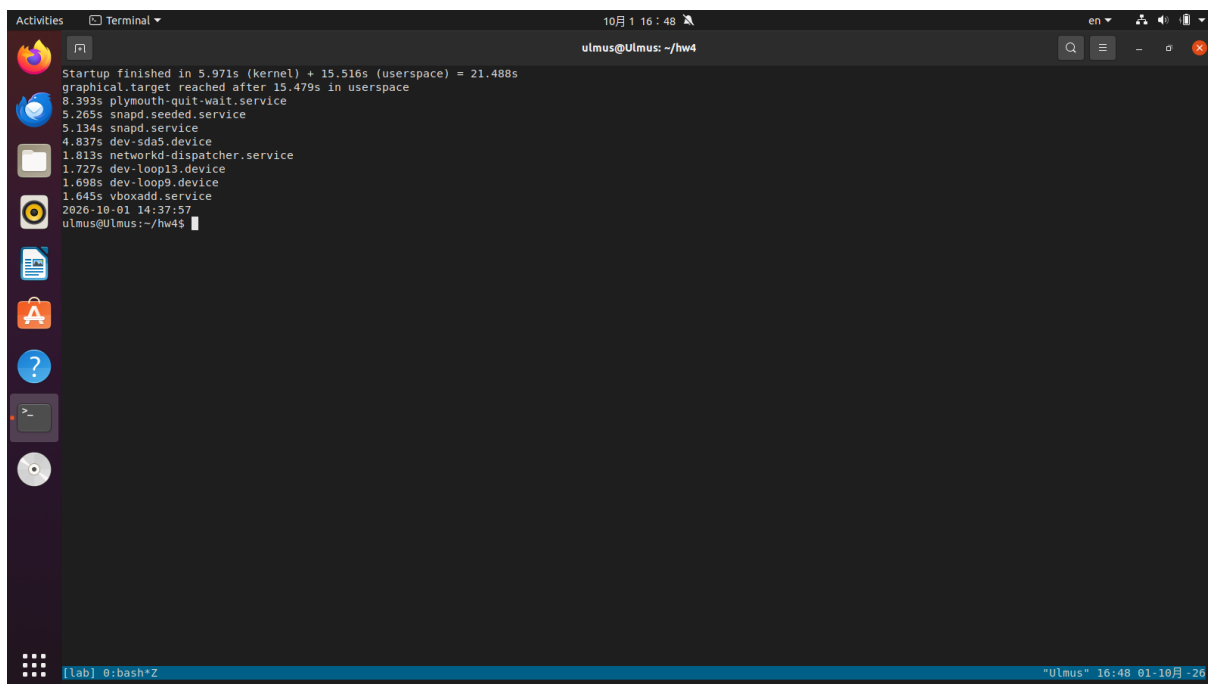


图 5-2 虚拟机引导耗时（systemd-analyze）

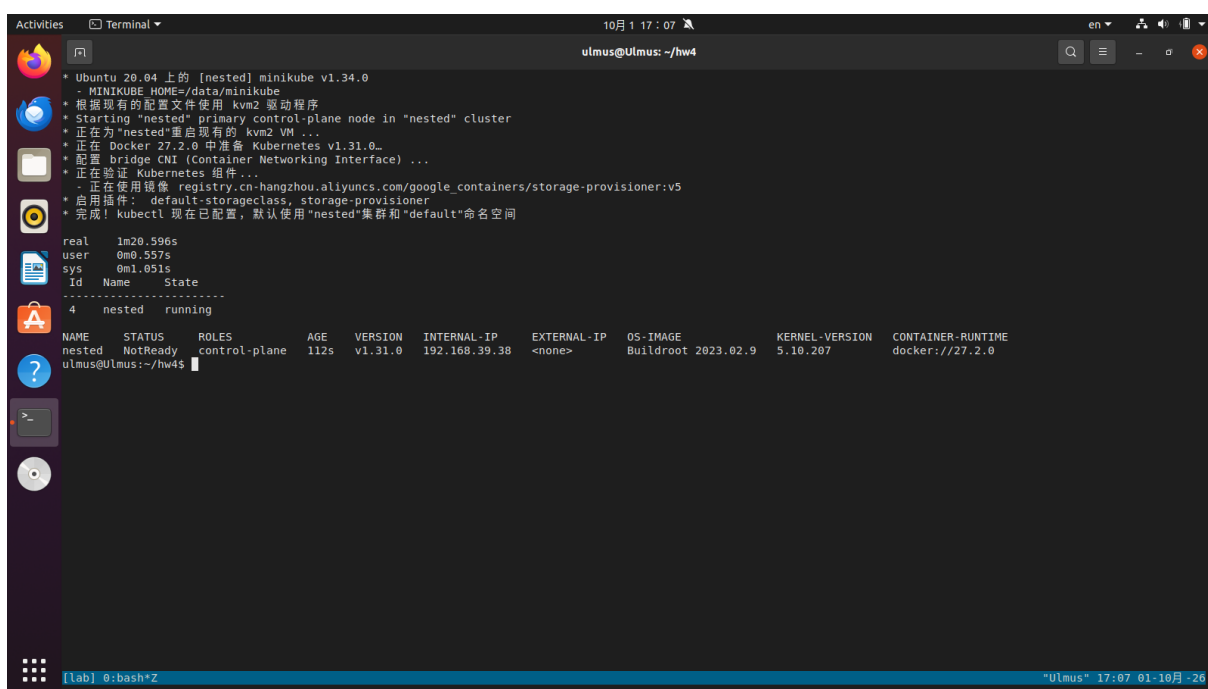


图 5-3 嵌套虚拟机中 minikube 的冷启动耗时

说明：minikube 节点的内核为 5.10.207，与外层虚拟机的 5.15 不同，直观说明虚拟机拥有独立内核。

数据呈现出非常清楚的数量级关系：进程是百毫秒级，容器是秒级，虚拟机是数十秒级。容器比进程多出的约 0.75 s 主要花在创建命名空间、cgroup、网络设备和 docker-proxy 上；Pod 比容器又多出约 0.5 s，来自调度、kubelet 同步和就绪探针的周期（periodSeconds=3）。真正的分水岭在虚拟机：无论是外层虚拟机的 21.5 s 还是嵌套

minikube 的 80.6 s，都要完整地引导一个操作系统内核。这也解释了为什么 HPA 扩容能在几秒内完成，而云厂商扩一个节点通常需要一两分钟。

5.3 资源占用

表 5-2 承载同一应用的内存开销

形态	测量方式	内存	说明
L0 物理机进程（waitress）	Windows 工作集	130.3 MB	gateway + backend 两个进程
L1 虚拟机进程（gunicorn）	ps RSS 求和	156.5 MB	6 个进程（2 master + 4 worker），共享页被重复计入
L2 compose 三个容器	docker stats	约 135 MiB	backend 65 + gateway 66 + Redis 3.4
L3 kind 空集群	free 增量	约 476 MB	控制面组件、kubelet、CNI、CoreDNS
L3 kind 含应用与插件	docker stats（节点容器）	1532 MiB	控制面 926 + worker2 480 + worker 126
L4 嵌套虚拟机	virsh domstats balloon.rss	2196 MiB	整个虚拟机的内存占用
L1 虚拟机本身	VirtualBox 分配	6 GB	空载已用约 985 MB（含 Docker 守护进程）

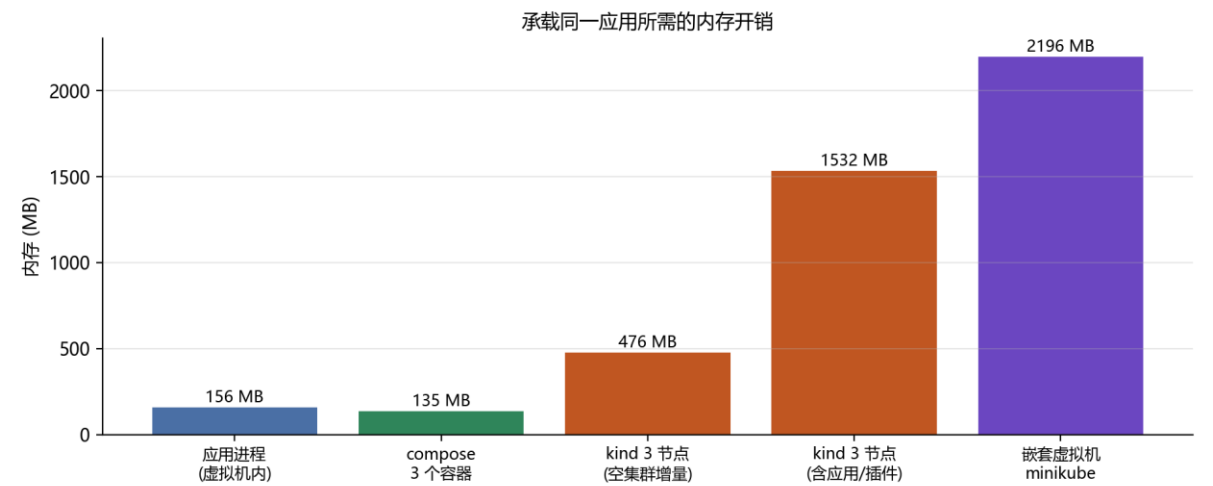


图 5-4 各层承载同一应用的内存开销

容器本身几乎不带来额外内存开销：三个容器约 135 MiB，与直接运行进程处于同一水平，因为容器只是受限的进程，不包含内核。开销主要来自更上层：K8s 控制面本身就要接近 1 GB（etcd、apiserver、controller-manager、scheduler 都在控制面节点上），而嵌套虚拟机仅为一个单节点集群就占用了 2.2 GB。对于 6 GB 的虚拟机来说，kind 集群与嵌套集群无法同时运行，测试嵌套路径时只能先停掉 kind 集群。

5.4 性能

```
hwt4-host-bench
Microsoft Windows [家版 中文版]
13th Gen Intel(R) Core(TM) i9-13900HX
15.7 GB RAM
PS E:\大四\云计算\作业四> Get-NetTCPConnection -State Listen -LocalPort 5090,8090 | Select-Object LocalAddress,LocalPort,OwningProcess | Format-Table -AutoSize

LocalAddress LocalPort OwningProcess
-----
127.0.0.1      8090      35880
127.0.0.1      5090      34916

PS E:\大四\云计算\作业四> curl.exe -s http://127.0.0.1:8090/api/whoami; ""
{"backend":"Ulmus","backend_node":"-", "gateway":"Ulmus","version":"1.0.0"}

PS E:\大四\云计算\作业四> .venv\Scripts\python.exe hwt4\scripts\bench.py http://127.0.0.1:8090/api/whoami -n 2000 -c 20 --label "Windows host native process (waitress)"
label      : Windows host native process (waitress)
requests   : 2000 (concurrency 20)
failed     : 0
elapsed    : 2.97 s
throughput : 672.6 req/s
latency ms : avg 29.54 | p50 29.35 | p95 34.88 | p99 36.52 | max 41.43
instances  :
  Ulmus      2000
PS E:\大四\云计算\作业四> "waitress working set: {0:N1} MB" -f ((Get-CimInstance Win32_Process | Where-Object { $_.CommandLine -match "waitress" } | ForEach-Object { (Get-Process -Id $_.ProcessId).WorkingSet64 } | Measure-Object -Sum).Sum/1MB)
waitress working set: 130.3 MB
PS E:\大四\云计算\作业四>
```

图 5-5 L0: 宿主机 Windows 上直接运行 (waitress)

说明: 在宿主机 PowerShell 中执行, 同时列出宿主机 CPU、内存及监听端口。

```
Activities Terminal 10月1 15:36
ulmus@Ulmus: ~/hwt4
ulmus@Ulmus:~/hwt4$ (cd src/backend 66 STORAGE_MODE=memory ~/venv-hwt4/bin/waitress-serve --listen=127.0.0.1:5090 app:app >/dev/null 2>&1 &); (cd src/gateway 66 BACKEND_U
RL=http://127.0.0.1:5090 ~/venv-hwt4/bin/waitress-serve --listen=127.0.0.1:8090 app:app >/dev/null 2>&1 &); sleep 2; ss -ltnp | grep -E ":(5090|8090)"; python3 scripts/b
ench.py http://127.0.0.1:8090/api/whoami -n 2000 -c 20 --label "VirtualBox VM native process (waitress)"; ps -o rss= -p $(pgrep -d, -f waitress-serve) | awk '{s+=\51} E
ND (printf "waitress RSS: %.1f MB\n", s/1024)'
LISTEN 0 1024 127.0.0.1:8090 0.0.0.0:* users:((("waitress-serve",pid=43772,fd=6))
LISTEN 0 1024 127.0.0.1:5090 0.0.0.0:* users:((("waitress-serve",pid=43775,fd=6))
label      : VirtualBox VM native process (waitress)
requests   : 2000 (concurrency 20)
failed     : 0
elapsed    : 3.53 s
throughput : 361.7 req/s
latency ms : avg 54.64 | p50 52.80 | p95 82.72 | p99 96.42 | max 135.49
instances  :
  Ulmus      2000
waitress RSS: 60.6 MB
ulmus@Ulmus:~/hwt4$
```

图 5-6 L1: 虚拟机内直接运行 (waitress, 与 L0 同服务器对照)

```
Activities Terminal 10月1 15:28 en 10月1 15:28 ulmus@Ulmus: ~/hw4
ulmus@Ulmus:~/hw4$ scripts/run_native.sh start; ps -eo pid,user,rss,cmd | grep -E "gunicorn.*(5090|8090)" | grep -v grep; curl -s localhost:8090/api/whoami; echo; python3 scripts/bench.py http://localhost:8090/api/whoami -n 2000 -c 20 --label "VM native process (gw x1, be x1)"
native gateway on :8090, backend on :5090
39620 ulmus 19536 /home/ulmus/venv-hw4/bin/python3 /home/ulmus/venv-hw4/bin/gunicorn -w 2 -b 0.0.0.0:5090 app:app
39622 ulmus 19484 /home/ulmus/venv-hw4/bin/python3 /home/ulmus/venv-hw4/bin/gunicorn -w 2 -b 0.0.0.0:8090 app:app
39625 ulmus 33552 /home/ulmus/venv-hw4/bin/python3 /home/ulmus/venv-hw4/bin/gunicorn -w 2 -b 0.0.0.0:8090 app:app
39627 ulmus 26124 /home/ulmus/venv-hw4/bin/python3 /home/ulmus/venv-hw4/bin/gunicorn -w 2 -b 0.0.0.0:5090 app:app
39629 ulmus 32440 /home/ulmus/venv-hw4/bin/python3 /home/ulmus/venv-hw4/bin/gunicorn -w 2 -b 0.0.0.0:8090 app:app
39630 ulmus 26156 /home/ulmus/venv-hw4/bin/python3 /home/ulmus/venv-hw4/bin/gunicorn -w 2 -b 0.0.0.0:5090 app:app
{"backend":"Ulmus","backend_node":"-","gateway":"Ulmus","version":"1.0.0"}
label : VM native process (gw x1, be x1)
requests : 2000 (concurrency 20)
failed : 0
elapsed : 2.77 s
throughput : 720.7 req/s
latency ms : avg 26.99 | p50 25.70 | p95 38.24 | p99 49.94 | max 59.98
instances : 2000
ulmus@Ulmus:~/hw4$
```

图 5-7 L1: 虚拟机内直接运行（gunicorn，作为 L2~L4 的基线）

```
Activities Terminal 10月1 15:24 en 10月1 15:24 ulmus@Ulmus: ~/hw4
ulmus@Ulmus:~/hw4$ docker compose ps --format "table {{.Name}}\t{{.Status}}"; python3 scripts/bench.py http://localhost:8080/api/whoami -n 2000 -c 20 --label "VM + dock
NAME STATUS
hw4-backend-1 Up 4 minutes (healthy)
hw4-gateway-1 Up 21 minutes (healthy)
hw4-redis-1 Up 21 minutes (healthy)
label : VM + docker compose (gw x1, be x1)
requests : 2000 (concurrency 20)
failed : 0
elapsed : 3.61 s
throughput : 554.1 req/s
latency ms : avg 35.53 | p50 34.76 | p95 46.65 | p99 55.15 | max 67.30
instances : 2000
8c2ee8da9277
NAME CPU % MEM USAGE / LIMIT NET I/O PIDS
hw4-backend-1 0.01% 65.14MiB / 5.783GiB 1.12MB / 1.32MB 3
hw4-gateway-1 6.14% 65.89MiB / 5.783GiB 2.69MB / 2.69MB 3
hw4-redis-1 3.07% 3.449MiB / 5.783GiB 17.5kB / 5.03kB 6
total used free shared buff/cache available
Mem: 5921 1138 1000 12 3782 4484
ulmus@Ulmus:~/hw4$
```

图 5-8 L2: docker compose 部署（经端口映射）

说明：同时给出 docker stats 与虚拟机内存。

```
Activities Terminal 10月1 16:41 en ulmus@Ulmus: ~/hw4

backend-856bf85cd7-gm6ks 1/1 Running hw4-worker2
gateway-6ff66f4968-5z498 1/1 Running hw4-worker2
redis-0 1/1 Running hw4-worker2
label : VM + kind K8s via NodePort (gw x1, be x1)
requests : 2000 (concurrency 20)
failed : 0
elapsed : 10.09 s
throughput : 198.2 req/s
latency ms : avg 100.27 | p50 98.73 | p95 153.72 | p99 171.90 | max 195.52
instances :
  backend-856bf85cd7-gm6ks 2000
NAME CPU(cores) CPU% MEMORY(bytes) MEMORY%
hw4-control-plane 177m 4% 923Mi 15%
hw4-worker 121m 3% 126Mi 2%
hw4-worker2 186m 4% 480Mi 8%
NAME CPU % MEM USAGE / LIMIT
hw4-worker2 4.23% 479.7MiB / 5.783GiB
hw4-control-plane 11.44% 926.1MiB / 5.783GiB
hw4-worker 1.60% 125.0MiB / 5.783GiB
Mem: total used free shared buff/cache available
ulmus@Ulmus:~/hw4$
```

图 5-9 L3: kind K8s 经 NodePort 访问（默认 CPU limit 500m）

说明：同时给出节点资源与节点容器的 docker stats。

```
Activities Terminal 10月1 16:43 en ulmus@Ulmus: ~/hw4

nr_periods 499
nr_throttled 415
throttled time 40240069273
NAME CPU_REQ CPU_LIMIT
gateway 100m 500m
backend 100m 500m
deployment.apps/gateway patched
deployment.apps/backend patched
Waiting for deployment "gateway" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "gateway" rollout to finish: 1 old replicas are pending termination...
deployment "gateway" successfully rolled out
deployment "backend" successfully rolled out
NAME CPU_REQ CPU_LIMIT
gateway 100m <none>
backend 100m <none>
label : VM + kind K8s via NodePort, no CPU limit
requests : 2000 (concurrency 20)
failed : 0
elapsed : 4.95 s
throughput : 403.7 req/s
latency ms : avg 48.89 | p50 48.48 | p95 60.05 | p99 64.74 | max 72.79
instances :
  backend-64fc86979c-wsv0l 2000
ulmus@Ulmus:~/hw4$
```

图 5-10 L3: 确认 CFS 节流并去掉 CPU limit 后复测

说明：gateway 的 cgroup 统计显示 499 个调度周期中有 415 个被节流；去掉 limit 后吞吐从约 200 提升到约 400 req/s。

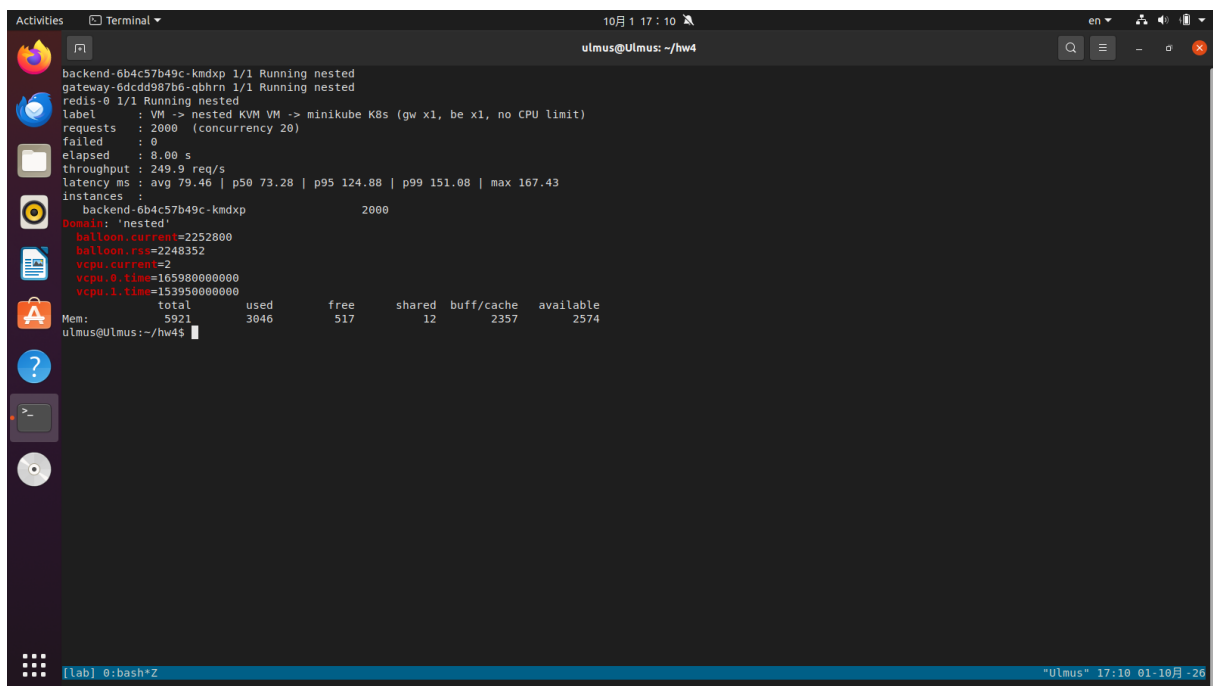


图 5-11 L4：嵌套虚拟机中的 minikube（无 CPU limit）

说明：同时给出 virsh domstats 采集的嵌套虚拟机内存与 vCPU 时间。

表 5-3 各层性能对比（/api/whoami，2000 请求，并发 20，3 次均值）

形态	吞吐 (req/s)	P50 (ms)	P95 (ms)	相对基线
L0 物理机进程（waitress）	617.2	32.1	38.0	waitress 基线
L1 虚拟机进程（waitress）	389.7	49.5	73.1	-36.9%
L1 虚拟机进程（gunicorn）	761.9	25.3	33.7	gunicorn 基线
L2 Docker（直连容器 IP）	651.0	29.9	37.5	-14.6%
L2 Docker（端口映射）	556.4	35.0	43.5	-27.0%
L3 kind（NodePort，无 limit）	422.5	46.2	59.6	-44.5%
L3 kind（Ingress，无 limit）	392.1	49.7	62.5	-48.5%
L3 kind（NodePort，limit 500m）	200.7	98.3	144.9	-73.7%
L4 嵌套 minikube（无 limit）	271.2	68.9	105.6	-64.4%

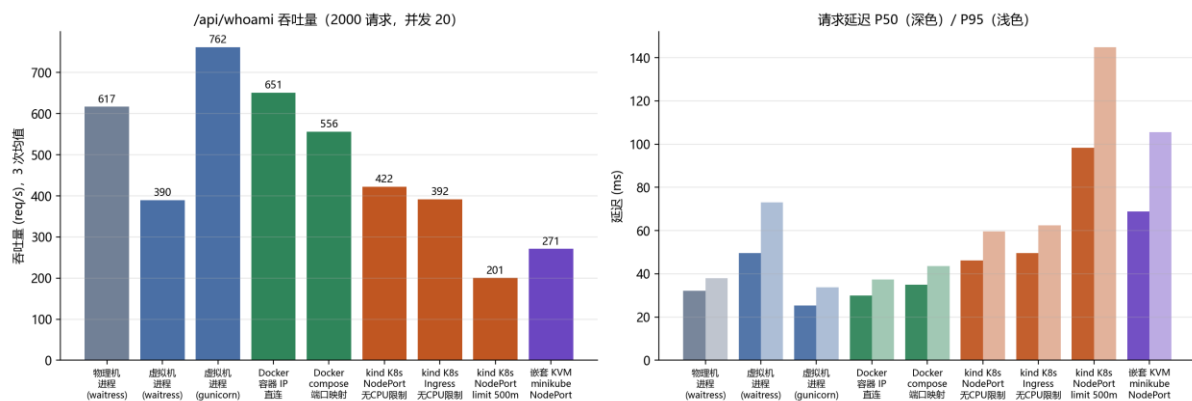


图 5-12 各层吞吐量与延迟对比

把这张表按“每次只增加一层”的方式拆开来看：

- **虚拟化层（L0→L1）**：同为 waitress，吞吐下降约 37%。宿主机有 32 个逻辑线程，虚拟机只有 4 个 vCPU，压测客户端与两个服务进程挤在一起，这部分差距更多来自“可用 CPU 变少”，而不完全是虚拟化指令开销。
- **容器层（L1→L2）**：直连容器 IP 时下降约 15%，来自 veth/网桥转发、Python 版本差异（3.8 vs 3.11）等；再经过端口映射又下降约 12 个百分点——docker-proxy 是用户态转发，每个连接多一次拷贝。也就是说，Docker 的网络开销有一半出在“为了让外部访问”的那一步。
- **编排层（L2→L3）**：在去掉 CPU limit、副本数相同的前提下，K8s 比 compose 再低约 24%。请求要经过端口映射到控制面节点容器、kube-proxy 的 iptables DNAT、kindnet 跨节点转发，gateway 到 backend 又要再走一次 ClusterIP。Ingress 多一跳 nginx，又低约 7%。
- **资源限制**：默认清单里的 limits.cpu=500m 让吞吐直接减半，这比编排层本身的网络开销大得多。limit 是 K8s 里最容易被忽视的性能因素。
- **嵌套虚拟化（L3→L4）**：嵌套虚拟机只有 2 个 vCPU，并且经过两层硬件虚拟化，吞吐约为 kind 的 64%，P95 延迟超过 100 ms。功能完全可用，但性能明显打折。

5.5 部署复杂度

表 5-4 从零到应用可访问的步骤与配置量

形态	需要编写的配置	有效行数	主要步骤
L1 虚拟机进程	run_native.sh	13	建虚拟环境 → 安装依赖 → 启动两个进程（Redis 需另装）
L2 docker compose	2 个 Dockerfile + compose + .env	84	安装 Docker 并配置镜像源 → compose up --build
L3 kind K8s	kind 配置 + 9 个清单（另需 Dockerfile）	29 + 325	安装 kubectl/kind → 建集群 → 导入镜像 → apply → 装 metrics-server → 装 ingress-nginx → 配 hosts
L4 嵌套 minikube	同 L3（不含 kind 配置）	325	装 libvirt/kvm2 驱动 → 预置 K8s 二进制 → minikube start → image load → apply

复杂度并不是线性增长的。从进程到 Docker，多出的是“镜像”这一个概念，换来的是环境一致性；从 Docker 到 K8s，配置量增加了约 4 倍，还多出了集群本身的搭建和插件安装。这些配置中真正描述“应用”的部分并不多，大量行数花在探针、资源、调度、中断预算等“运行策略”上——这些正是 compose 不具备的能力。

5.6 隔离性

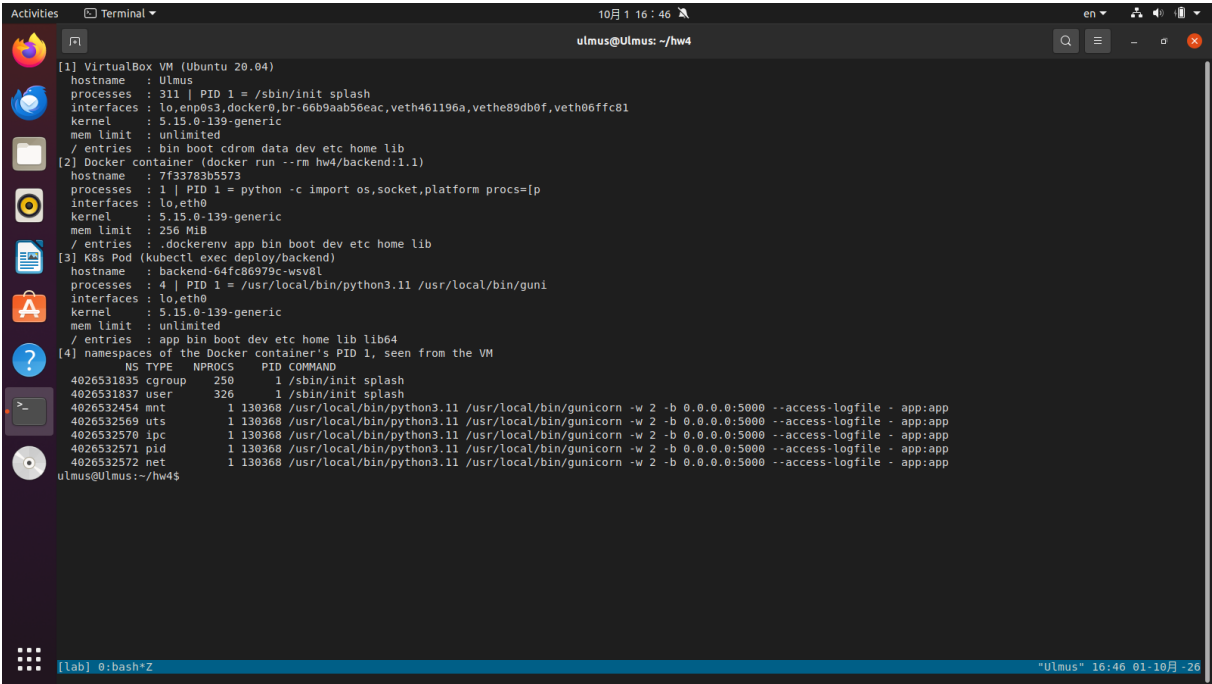


图 5-13 虚拟机、Docker 容器与 K8s Pod 的隔离视图对比

说明：同一脚本分别在虚拟机、容器（限制 256 MiB 内存）与 Pod 中执行，并从虚拟机侧用 lsns 查看容器进程的命名空间。

表 5-5 隔离程度对比

维度	虚拟机	Docker 容器	K8s Pod
进程视图	看到全部 311 个进程，PID 1 为 systemd	只看到自己（1 个），PID 1 为应用	4 个（gunicorn master 与 worker）
主机名	Ulmus	容器 ID	Pod 名
网络接口	物理网卡 + docker0 + 多个 veth	仅 lo、eth0	仅 lo、eth0
根文件系统	完整 Ubuntu	镜像文件系统（含 .dockerenv）	镜像文件系统
内核	5.15.0-139（自有）	5.15.0-139（与虚拟机共享）	5.15.0-139（共享）
内存限制	无（由 VirtualBox 分配 6 GB）	cgroup 限制 256 MiB	由 limits 决定
命名空间	—	独立 mnt/uts/ipc/pid/net；共享 cgroup、user	同容器，且 Pod 内容容器共享 net

隔离对比最直接的结论是“内核只有一个”。容器、Pod 的 uname 与虚拟机完全相同，而嵌套虚拟机里的 minikube 节点是另一个内核版本（5.10.207）。容器的隔离是命名空间和 cgroup 给出的“视图隔离”与“资源限制”，lsns 显示它甚至与宿主共享 user 命名空间，容器内的 root 就是宿主的 root；虚拟机则在硬件虚拟化之上运行独立内核。

这意味着两者的安全边界不同：多租户、不可信代码或需要不同内核的场景应该用虚拟机，同一团队的服务之间用容器足够。

5.7 嵌套虚拟化

VirtualBox 开启“嵌套 VT-x/AMD-V”后，虚拟机内 /proc/cpuinfo 出现 vmx 标志，kvm-ok 确认可以使用 KVM 加速。在此基础上用 minikube 的 kvm2 驱动成功创建了第二层虚拟机并运行 K8s，应用部署与访问都正常。但过程并不顺利：第一次创建时嵌套虚拟机引导过慢，超过了 minikube 等待 IP 的 1 分钟时限；之后冷启动耗时 80.6 s，是 kind 创建三节点集群的 1.75 倍，压测性能也只有 kind 的 64%。嵌套虚拟化可行，但它更适合用来做实验和验证，不适合承载需要性能的业务。

5.8 结论：何时选虚拟机，何时选容器

表 5-6 各层的收益与代价

层次	解决的问题	付出的代价（本次实测）
虚拟化	独立内核与强隔离；可在 Windows 上运行 Linux；资源可按虚拟机划分	引导 21.5 s；同服务器吞吐 -37%（含 vCPU 变少）；6 GB 内存需预先分配
容器	环境一致、秒级启动、进程级隔离与资源限制	启动 +0.75 s；网络 -15%~-27%；需要学习镜像与网络模型
编排	声明式运维：自愈、扩缩容、滚动发布、服务发现、配置管理、多节点容错	控制面约 1 GB 内存；网络再 -24%；配置量约 4 倍；探针等配置不当会制造故障
嵌套虚拟化	在没有物理机的条件下模拟“虚拟机上跑 K8s 节点”	冷启动 80 s；性能为 kind 的 64%；内存 2.2 GB

如果只用一句话总结：需要不同内核、强安全边界或整机交付时选虚拟机；需要快速交付、密集部署同类服务时选容器；当服务需要多副本长期在线、频繁发布、自动伸缩时，再在容器之上叠加 K8s。在云上这三层往往同时存在——云厂商用虚拟机切分物理机，K8s 节点就是这些虚拟机，业务以容器的形式跑在节点上。

六、问题与分析

下面按影响程度从环境问题到设计问题依次记录。每个问题都给出现象、排查过程与解决方法，其中问题 4~8 是在故障注入实验中才暴露出来的设计缺陷，对理解 K8s 的帮助最大。

问题 1：虚拟机内无法拉取 Docker Hub 镜像

现象：虚拟机能访问 GitHub、dl.k8s.io，但访问 registry-1.docker.io 超时，docker pull 失败。

排查：用 curl 分别测试 /v2/ 接口，确认是 Docker Hub 不可达而非 DNS 问题；再逐个测试国内镜像站，选出返回 401（说明可达）且延迟最低的几个。

解决：在 /etc/docker/daemon.json 中配置 registry-mirrors；kind 节点内部用的是 containerd，不读 Docker 的配置，因此在 kind-cluster.yaml 的 containerdConfigPatches 中为 docker.io、registry.k8s.io、quay.io 单独配置镜像源，metrics-server 与 ingress-nginx 的镜像才得以拉取。

问题 2：虚拟机磁盘空间不足

现象：虚拟机系统盘仅剩 8.9 GB，而 kind 节点镜像约 1 GB，加上应用镜像与嵌套虚拟机磁盘，空间明显不够。

解决：在宿主机 E 盘新建 40 GB 虚拟磁盘挂到虚拟机，格式化后挂载为 /data，并通过 daemon.json 的 data-root 把 Docker 数据目录放到 /data/docker，minikube 的 MINIKUBE_HOME 也设在 /data 下。系统盘未做任何改动。

问题 3：单容器测试中的两个细节

现象一：接口返回的中文错误信息变成 \u8d85\u8fc7 形式。原因是 Flask 的 JSON 默认 ensure_ascii=True，设置 app.json.ensure_ascii = False 后正常显示（图 4-3 为修正后的结果）。

现象二：想用 docker exec 容器 kill 1 模拟进程崩溃，提示找不到 kill 命令——slim 镜像不含 procp。改为在容器内用 Python 的 os.kill 向 PID 1（gunicorn master）发送 SIGTERM，gunicorn 正常退出后由重启策略拉起，完成了实验（图 4-34）。这让我注意到精简镜像里连基本的进程工具都没有，排障手段要提前考虑；同时容器的生命周期就是 PID 1 的生命周期，应用必须正确处理 SIGTERM。

问题 4：就绪探针依赖 Redis，导致级联摘流

现象：删除 redis-0 后，持续探测中 105 次请求失败 47 次，而失败的恰恰是完全不需要 Redis 的 /api/whoami，错误为 Connection refused（图 4-25）。

排查：Connection refused 来自 gateway 访问 backend Service 时，而不是 Redis。
kube-proxy 对没有任何就绪后端的 Service 会直接拒绝连接，结合 v1.0 的代码可以确定原因：/readyz 会 ping Redis，Redis 一重建，3 个 backend 同时被判定为未就绪，被一起从 Endpoints 中摘除，gateway 自然连不上任何后端；Redis 恢复后，各 backend 要再经过几个探测周期才重新就绪，所以中断比 Redis 自身的重建更长。

解决：就绪探针只应回答“这个实例能不能接流量”，不应检查共享依赖。v1.1 中 /readyz 只反映自身状态，依赖状态改由 /api/deps 暴露，Redis 异常时用到存储的接口快速返回 503。复测时 /api/whoami 164 次请求全部成功，Endpoints 始终保持 3 个地址（图 6-1）。

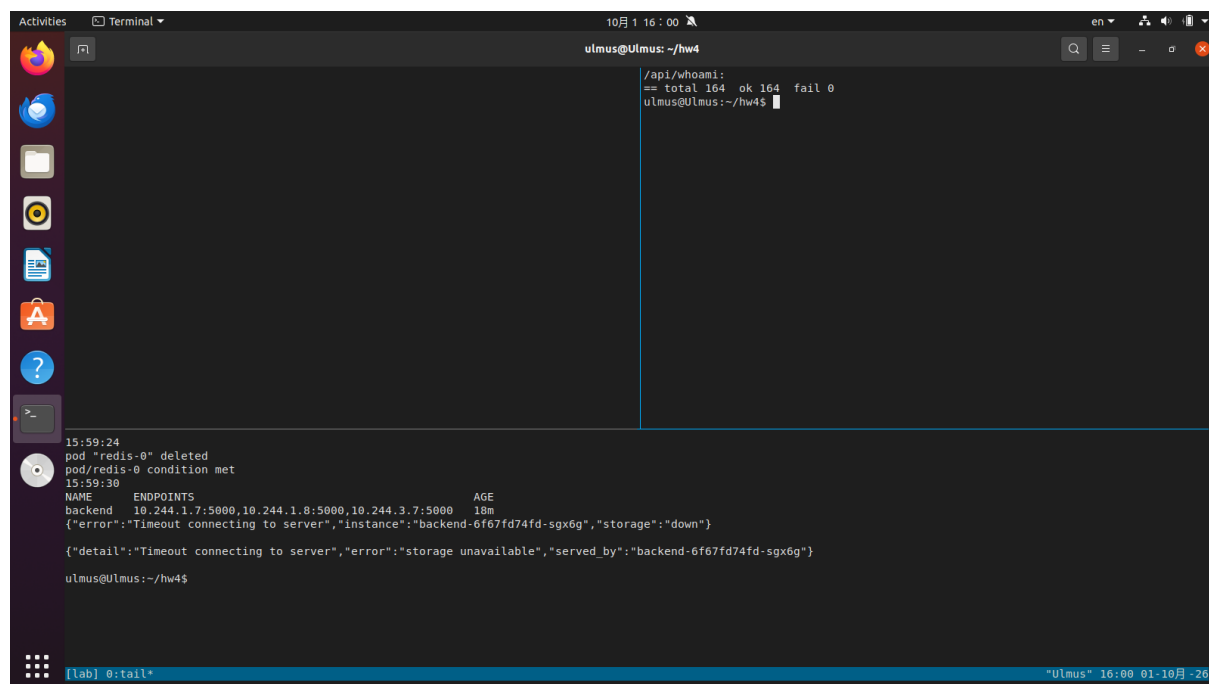


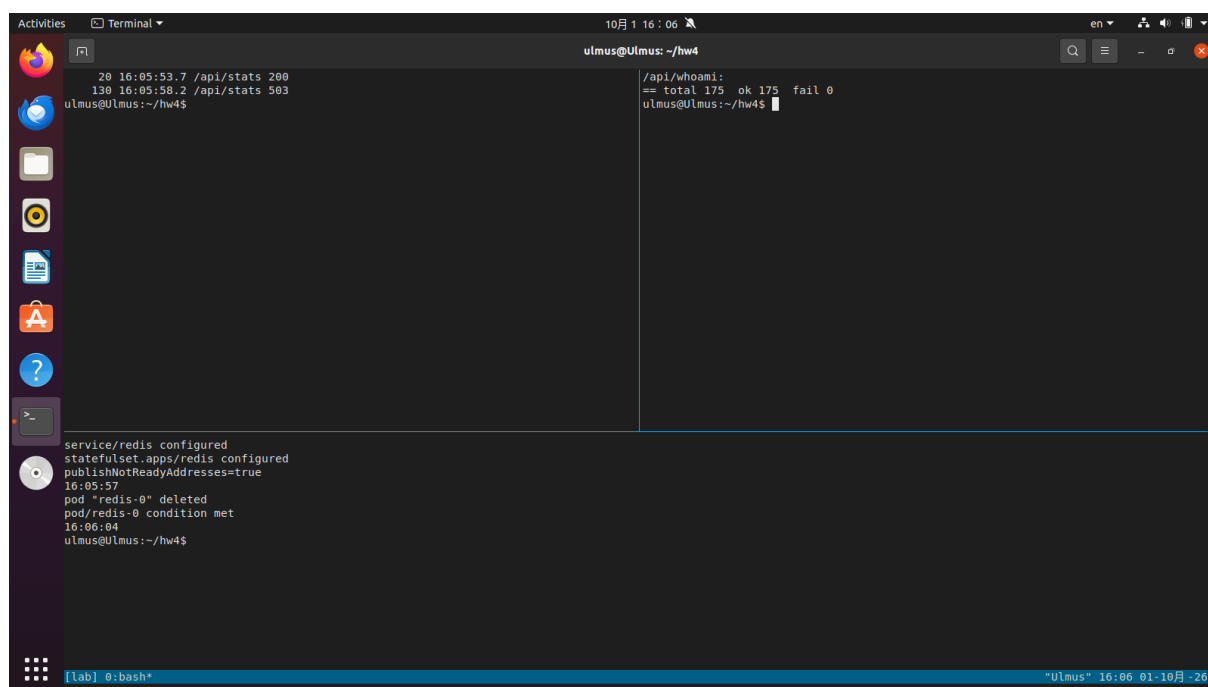
图 6-1 v1.1 中 Redis 重建期间 /api/whoami 不受影响

问题 5：Redis 已就绪，业务仍长时间连不上

现象：修正问题 4 后再测，发现存储接口的不可用时间约 29 s，而 Redis 本身 6 s 就已 Ready，相差二十多秒。

排查：确认 Redis 数据完好、从 Pod 内用新建客户端可以 ping 通，问题在于“找到”Redis。业务通过 redis-0.redis 访问，这个名字解析到的是 Pod IP，Pod 重建后 IP 会变；查看 CoreDNS 配置发现 ttl 30、cache 30。先尝试给 Headless Service 加 publishNotReadyAddresses，用脚本精确测得不可用窗口仍有 24.7 s（Ready 后仍有 18.7 s），说明主要不是 NXDOMAIN 被缓存，而是旧 IP 被缓存（图 6-2）。

解决：增加一个普通 ClusterIP Service redis-master 供业务访问。ClusterIP 是稳定的虚拟 IP，Pod 重建只需 kube-proxy 更新后端，不受 DNS 缓存影响。复测不可用窗口降到 6.0~6.6 s，且恰好在 Redis Ready 时结束（图 6-3）。Headless Service 继续保留，用于 StatefulSet 的稳定网络标识。



```
Activities Terminal 10月1 16:06 en
ulmus@Ulmus: ~/hw4

20 16:05:53.7 /api/stats 200
130 16:05:58.2 /api/stats 503
ulmus@Ulmus:~/hw4$

/api/whoami:
== total 175 ok 175 fail 0
ulmus@Ulmus:~/hw4$

service/redis configured
statefulset.apps/redis configured
publishNotReadyAddresses=true
16:05:57
pod "redis-0" deleted
pod/redis-0 condition met
16:06:04
ulmus@Ulmus:~/hw4$

[Lab] 0: bash* "Ulmus" 16:06 01-10月-26
```

图 6-2 仅加 publishNotReadyAddresses 时存储接口仍长时间 503

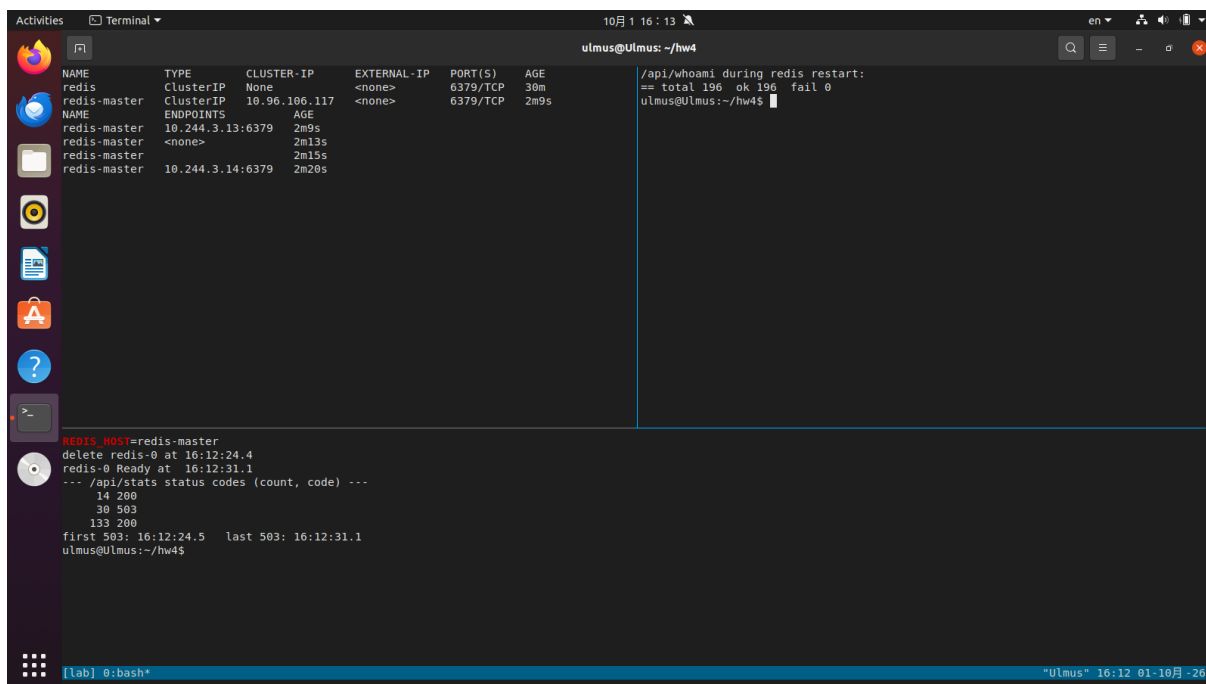


图 6-3 改用 ClusterIP Service 后存储接口在 Redis 就绪时即恢复

表 6-1 Redis 重建期间存储接口不可用时间

方案	Redis Ready 耗时	存储接口不可用窗口	Ready 之后的额外等待
直连 redis-0.redis（原方案）	约 6 s	约 29 s	约 24 s
+ publishNotReadyAddresses	6.3 s	24.7 s	18.7 s
ClusterIP Service redis-master	6.4~6.7 s	6.0~6.6 s	≈ 0

问题 6：缩容时偶发 502

现象：从6副本缩到2副本时，108次请求中出现1次502（图4-21）。

排查：删除 Pod 时，kubelet 向容器发送 SIGTERM 与 Endpoints 控制器、kube-proxy 更新转发规则是并行进行的。gunicorn 收到 SIGTERM 后立即停止接受新连接，而 kube-proxy 可能还没把它摘掉，于是有请求被转发到一个已经拒绝连接的实例。

解决: 为 gateway 与 backend 增加 preStop: sleep 5, 先让 Pod 从 Endpoints 中摘除、在途请求处理完, 再发送 SIGTERM。复测 128 次请求全部成功 (图 4-22)。这个问题只在单副本或低频访问时不明显, 持续探测才把它暴露出来。

问题 7：PDB 与 HPA 冲突，节点永远无法排空

现象：kubectl drain 时，gateway 被正常迁移，backend 一直报“would violate the pod's disruption budget”，直到 120 s 超时（图 4-29）。

排查：此时 HPA 已把 backend 缩到最小值 2，而 PDB 写的是 minAvailable: 2。允许中断数 = 当前健康数 2 - 最少可用 2 = 0，任何 Pod 都不能被驱逐。两个配置单独看都合理，组合在一起就形成了死锁。

解决：把 backend 的 PDB 改为 maxUnavailable: 1，它与副本数无关，始终允许逐个迁移。修改后 drain 在 7 s 内完成，期间 173 次请求全部成功（图 4-30）。经验是：PDB 应当和副本数、HPA 的最小值一起设计。

问题 8：CPU limit 让吞吐减半

现象：第一次在 K8s 上压测只有约 200 req/s，不到 compose 的 40%。

排查：清单中 gateway 与 backend 都设置了 limits.cpu: 500m。读取 Pod 的 cgroup cpu.stat，压测期间 499 个调度周期中 415 个被节流（图 5-10）。

解决：对比实验中去掉 limit 后吞吐回到约 420 req/s。最终清单仍保留 limit——在多租户集群中它是防止单个服务拖垮节点的必要手段——但应根据压测结果来定值，而不是随手写一个。

问题 9：节点宕机后约 40 秒的请求超时

现象：直接停止一个工作节点后，约 38 s 内有 21 次请求超时（图 4-31）。

分析：节点宕机时 kubelet 已经无法上报状态，API 中该节点上的 Pod 仍显示 Running 1/1。只有当节点控制器在 grace period（约 40 s）后把节点标记为 NotReady，Pod 的就绪状态才被置为 False 并从 Endpoints 中摘除。与 drain 这种“计划内中断”不同，K8s 对非计划中断的感知是有延迟的。

改进思路：可以缩短节点监控周期，但会增加误判；更实用的是在调用方做短超时 + 幂等请求重试，让单个后端失效不直接暴露给用户。这也说明自愈不等于零中断。

问题 10：嵌套虚拟机中 minikube 启动失败

现象一：首次创建报“machine nested didn't return IP after 1 minute”。查看 libvirt 的 DHCP 租约发现虚拟机其实拿到了 IP，只是引导太慢，超出了等待时限。删除后重建（ISO 已缓存）即可在时限内完成。

现象二：使用 `--image-mirror-country=cn` 时，从镜像站下载 kubectl 的校验文件返回 404。由于 dl.k8s.io 在虚拟机中可直接访问，手动下载 kubeadm、kubect、kubectl 放入 minikube 缓存目录后启动成功。

问题 11：其他值得记录的现象

rollout 完成 ≠ 流量已切换：修改 ConfigMap 并 rollout restart 后立即请求，仍拿到旧标题（图 4-36）。因为旧 Pod 正处在 preStop 的 5 s 等待中，kubectl rollout status 只关心新副本是否就绪。验证变更时需要留出这段时间。

集群整体重启后的短暂 DNS 故障：停止再启动三个 kind 节点后，CoreDNS 有一段时间无法访问 10.96.0.1（apiserver 的 ClusterIP），业务报域名解析失败；等 kube-proxy 重建规则、CoreDNS 恢复后自愈。

七、心得体会与技能收获

7.1 我是什么时候真正分清容器和虚拟机的

之前我对“容器共享内核”的理解只停留在概念上，真正想通是在做隔离性对比的时候。同一段脚本在虚拟机、容器和 Pod 里打印出来的内核版本一模一样，都是 5.15.0-139；容器里只能看到 1 个进程，PID 1 就是我的 Python 程序；而在嵌套虚拟机里，minikube 节点的内核变成了 5.10.207。那一刻我意识到，容器并没有“虚拟”出任何硬件，它只是让一个普通进程戴上了一副“眼镜”，看不到别的东西而已。后来做故障实验时，容器的生命周期随 PID 1 结束而结束、lsns 显示容器与宿主共享 user 命名空间，这些细节都在印证同一件事：容器是进程，虚拟机是机器。启动时间上 0.19 s、0.95 s 与 21.5 s 的数量级差异，也是这个本质区别的直接结果。

7.2 K8s 带来的麻烦和便利

便利是非常直观的：删一个 Pod，3 秒后新的就顶上了，303 次请求一个没丢；压测一开，HPA 半分钟内把副本从 2 扩到 6，压测结束又自己缩回去；滚动更新、节点排空，全程用户无感。这些在 `compose` 里要么做不到，要么得自己写脚本。

麻烦也同样真实，而且比我预想的多。这次遇到的几个最有价值的问题——就绪探针级联摘流、DNS 缓存、缩容 502、PDB 死锁、CPU 节流——全部是 K8s 特有的，而且没有一个会让部署失败，它们只会在故障或压力下突然出现。K8s 给了很多旋钮，每个旋钮单独看都合理，组合在一起却可能互相打架，比如 PDB 的 `minAvailable` 和 HPA 的 `minReplicas`。我的体会是，K8s 把“怎么运行”变成了代码，那这部分代码也会有 bug，也需要测试。

所以我认为 K8s 适合的场景是：服务数量较多、需要多副本长期在线、发布频繁、负载有波动，并且团队愿意投入精力理解它的运行机制。像这次这样的单机小应用，用 `compose` 就足够，硬上 K8s 只会多出近 1 GB 的控制面内存和 4 倍的配置量。

7.3 对“云原生”的新理解

做实验之前，我以为“云原生”就是“用 Docker 和 K8s”。现在我觉得它更像是一种对应用的要求：应用要假设自己随时可能被杀掉、被复制、被搬走。为了让我的留言板在 K8s 里表现良好，我改的其实大多是应用本身——让实例标识来自环境变量、让配置全部外置、把存活和就绪区分开、让依赖故障时快速失败而不是挂起、让进程正确处理 `SIGTERM`。工具只是提供了机制，应用要按这些机制的规则来写，才能真正享受到自愈和弹性。这也解释了为什么同样跑在 K8s 上，有的系统很稳定，有的却频繁出问题。

7.4 如果重做一次

- 先做故障注入，再写探针和中断预算。这次是先按常见写法配好，再被实验结果一一推翻；如果一开始就带着持续探测去设计，能少走不少弯路。
- 从一开始就把 Redis 的访问地址设计成 `ClusterIP Service`，而不是直接用 `StatefulSet` 的 Pod 域名；有状态服务的“稳定标识”和“稳定访问入口”是两回事。
- 在 `gateway` 中加入短超时与对幂等请求的一次重试，以覆盖节点宕机后约 40 s 的摘流延迟。

- 压测客户端放到另一台机器上，避免与服务端争用虚拟机仅有的 4 个 vCPU，让各层性能差异更纯粹。
- Redis 用 Sentinel 或主从结构，使存储层也具备故障转移能力，而不是只能等单个 Pod 重建。

7.5 技能收获与仍存的疑问

技能上，我系统地练习了 Dockerfile 分层与安全实践、compose 编排、kind 多节点集群、ConfigMap/Secret/Downward API、探针、HPA、PDB、StatefulSet、Ingress、滚动更新与节点排空，以及用 cgroup、lsns、virsh 等工具从底层验证现象。更重要的是养成了“用数据说话”的习惯：每个结论都尽量有一次可复现的测量支撑。

仍存的疑问有两个：一是 kind 的节点本身是容器，它测出的网络开销与真实多机集群（跨物理网络、使用 overlay 网络插件）相比有多大差距；二是在 Pod 内存限制与 Python 进程的内存行为之间，如何科学地设置 requests 与 limits，这次只验证了 CPU limit 的影响，内存方面还没有深入。

八、参考文献与资料

- [1] Docker Documentation. Dockerfile reference / Compose file reference / Configure registry mirrors. <https://docs.docker.com/>
- [2] Kubernetes Documentation. Deployments; StatefulSets; Services (Headless Services); ConfigMaps; Secrets; Downward API. <https://kubernetes.io/docs/>
- [3] Kubernetes Documentation. Configure Liveness, Readiness and Startup Probes; Container Lifecycle Hooks (preStop); Pod Termination.
- [4] Kubernetes Documentation. Horizontal Pod Autoscaling; Specifying a Disruption Budget for your Application; Safely Drain a Node.
- [5] Kubernetes Documentation. Resource Management for Pods and Containers (CPU limit 与 CFS quota) .
- [6] kind Documentation. Configuration (extraPortMappings、containerdConfigPatches) ; Ingress; Loading an Image. <https://kind.sigs.k8s.io/>
- [7] minikube Documentation. KVM2 driver; Start options (--image-mirror-country) . <https://minikube.sigs.k8s.io/>
- [8] metrics-server 项目文档 (--kubelet-insecure-tls) . <https://github.com/kubernetes-sigs/metrics-server>
- [9] ingress-nginx Deployment Guide (kind provider) . <https://kubernetes.github.io/ingress-nginx/>
- [10] CoreDNS Manual. kubernetes 与 cache 插件. <https://coredns.io/manual/>
- [11] Oracle VM VirtualBox User Manual. Nested Virtualization; VBoxManage controlvm. <https://www.virtualbox.org/manual/>

- [12] Linux man-pages. namespaces(7), pid_namespaces(7), cgroups(7), lsns(8).
- [13] Flask Documentation. JSON Support (ensure_ascii) ; gunicorn Documentation. Signal Handling.